



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación 1

Reporte de Actividades — Semana 3

Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje de Gran Escala (LLM).

*Base de datos MongoDB, extracción masiva de texto plano,
exploración de parafraseo y detección ICL con LLMs*

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Mayo 2026

5,060 PDFs procesados — MongoDB activa — CBI 97.9% — 11 experimentos — BETO

F1=0.893 (PAWS-X)

Resumen Ejecutivo

En una página: lo esencial de la Semana 3

Objetivo: implementar la base de datos del corpus, completar la extracción PDF→texto de la división CBI y experimentar con LLMs y técnicas de *In-Context Learning* (ICL) para detección de paráfrasis en español.

Lo realizado

- **Persistencia:** MongoDB 7 sobre Docker con **21 284** registros indexados; pipeline reanudable validado con bug real en producción.
- **Extracción:** pipeline PyMuPDF + Tesseract sobre **5 060 PDFs** de CBI con **97.9 % de éxito** en ~6 h; texto plano completo almacenado en MongoDB.
- **Investigación:** catálogo de 15+ modelos en Hugging Face, datasets PAWS-X y STS-ES, revisión de 5 papers relevantes (COLING 2025, NAACL 2025). Reporte separado de 35 pp.
- **11 experimentos** de detección de paráfrasis cubriendo cuatro enfoques: embeddings (MiniLM), LLM local (11ama3.1:8b), LLM remoto (Groq Llama 3.3 70B) y fine-tuning supervisado (BETO).
- **Fine-tuning de BETO** en Google Colab T4 (**45 min**) sobre PAWS-X (49 401 pares).

Métricas clave

Resultado	Dataset	F1
MiniLM-L12-v2 (sin entrenamiento)	PAWS-X (500)	0.6584
Llama 3.3 70B con ICL (Groq)	PAWS-X (50)	0.7931
11ama3.1:8b con ICL graduado	Propio (39)	0.8571
BETO finetuned en PAWS-X	PAWS-X (1 907)	0.8925
<i>Luna 2022 (línea base, corpus propio)</i>	<i>CBI-M-00-21</i>	<i>0.970</i>

Hallazgo central

BETO fine-tuneado y llama3.1:8b con ICL cometen errores ortogonales en el mismo dataset: BETO tiene *precision* = 1.0 (cero falsos positivos) y llama3.1:8b tiene *recall* = 1.0 (cero falsos negativos). Un ensamble booleano sencillo (AND u OR) debería superar a cualquiera de los dos modelos por separado — esta es la primera línea de trabajo propuesta para la Semana 4.

Cómo está organizado este reporte

Parte 1 cubre la construcción del corpus (MongoDB + extracción). **Parte 2** contiene la investigación de modelos, datasets y estado del arte. **Parte 3** describe los 11 experimentos uno por uno. **Parte 4** consolida resultados, hace el contraste cuantitativo y cualitativo entre enfoques y enumera los candidatos a próximos pasos para presentar al asesor.

Los anexos finales incluyen el listado completo de scripts, índice de figuras y de tablas, y el árbol del repositorio.

Índice

Resumen Ejecutivo	1
I Construcción del Corpus	10
1. Introducción y Contexto del Proyecto	10
1.1. Objetivo general	10
1.2. Antecedentes (Semanas 1 y 2)	10
1.3. Objetivos de la Semana 3	10
1.4. Stack tecnológico	11
1.5. Estructura del trabajo	11
2. Prueba Piloto de Extracción (50 PDFs)	12
2.1. Motivación	12
2.2. Lógica del pipeline de extracción	12
2.3. Resultados de la prueba piloto	12
3. Base de Datos No Relacional con MongoDB	13
3.1. Justificación de la elección tecnológica	13
3.2. Instalación con Docker	13
3.3. Esquema del documento MongoDB	13
3.4. Índices creados	14
4. Scripts e Implementaciones	15
4.1. config.py — Configuración central	15
4.2. 01_cargar_indice.py — Carga inicial de la base de datos	15
4.3. 02_extraer_corpus.py — Pipeline masivo de extracción	16
4.3.1. Bug crítico corregido durante la ejecución	17
4.4. 03_stats_bd.py — Estadísticas y monitoreo	17
4.5. 04_exportar_json.py — Exportación del corpus a JSON	17
4.6. 05_explorar_parafrasis.py — Exploración de parafraseo con LLM	18
4.7. 06_deteccion_icl.py — Detección ICL (Experimento 1)	18
4.8. 07_icl_dificil.py — Dataset difícil (Experimento 2)	19
5. Secuencia de Ejecución	20

6. Resultados Finales de la Extracción CBI	21
6.1. Resumen ejecutivo	21
6.2. Estadísticas del texto extraído	21
7. Análisis de PDFs Problemáticos	22
7.1. Tipo A — Árbol de páginas corrupto (5 PDFs)	22
7.2. Tipo B — Número de páginas inválido (5 PDFs)	22
7.3. Tipo C — Error de capitalización (3 PDFs recuperables)	22
7.4. PDFs vacíos (93 documentos)	23
II Investigación de Modelos y Estado del Arte	27
8. Fundamentos Teóricos: Aprendizaje en Contexto (ICL)	27
8.1. Definición y Origen	27
8.2. Modalidades de ICL	27
8.3. ICL aplicado a Detección de Paráfrasis	28
9. Catálogo Exhaustivo de Modelos en Hugging Face	30
9.1. Categoría 1: Modelos de Embeddings para Similitud Semántica	30
9.1.1. Análisis Detallado de los Modelos de Embeddings más Relevantes	31
9.2. Categoría 2: Modelos NLI para Clasificación Cero-Shot	32
9.3. Categoría 3: LLMs Generativos con ICL Verdadero	33
9.3.1. Qwen 2.5 7B Instruct	34
9.3.2. Meta Llama 3.1 8B Instruct	34
9.3.3. Velocidad estimada en CPU (Intel i7-4770HQ, sin GPU)	35
9.4. Categoría 4: Modelos Base para Fine-Tuning en Español	35
10. Datasets para Paráfrasis en Español	37
10.1. PAWS-X Spanish (El Benchmark Estándar)	37
10.2. STS-ES (Similitud Semántica Textual en Español)	37
11. Estado del Arte: Revisión de Papers Relevantes	39
11.1. PARAPHRASUS: Benchmark Multi-dimensional (COLING 2025)	39
11.2. PlagBench: Plagio por Paráfrasis con LLMs (NAACL 2025)	39
11.3. PLD-4: Marco Multi-tarea para Paráfrasis de LLM (ACL ARR 2025)	40
11.4. Erosión de Firmas en Texto Científico (arXiv 2512.05311)	40
11.5. SciBERT: BERT para Texto Científico (EMNLP 2019)	40

12. Análisis de Infraestructura Disponible	41
12.1. Opciones de Ejecución según la Infraestructura	41
12.1.1. Opción A: Sentence-Transformers en CPU (Local, Sin GPU)	41
12.1.2. Opción B: API de Groq (Llama 3 70B, Gratuito)	42
12.1.3. Opción C: LLM Cuantizado con Ollama (Local, CPU)	42
12.1.4. Opción D: Fine-Tuning de BETO en Google Colab	43
13. Comparativa Final de Modelos	45
14. Estrategia de Implementación Recomendada	47
14.1. Pipeline Híbrido de Tres Etapas	47
14.2. Plan de Acción por Fases	47
14.2.1. Fase 1 — Prototipado con Embeddings	47
14.2.2. Fase 2 — Evaluación con LLM ICL	48
14.2.3. Fase 3 — Comparación y Documentación	48
15. Modelos No Recomendados para Este Proyecto	49
16. Conclusiones de la Investigación sobre LLMs e ICL	50
III Experimentos de Detección de Paráfrasis	51
17. Exploración de Parafraseo con LLM (llama3.1:8b)	51
17.1. Configuración del experimento	51
17.2. Resultados de similitud coseno	51
17.3. Observaciones del experimento	52
18. Experimento 1 — Detección ICL con llama3.1:8b	54
18.1. Descripción	54
18.2. Dataset	54
18.3. Ejecución del experimento	54
18.4. Resultados	55
18.5. Análisis crítico	55
19. Diagramas del Sistema	56
19.1. Diagrama de Arquitectura del Sistema	56
19.2. Esquema visual del Documento MongoDB	57

20.Experimento 2 — ICL con Dataset de Dificultad Graduada	58
20.1. Motivación	58
20.2. Construcción del dataset	58
20.3. Resultados finales	59
20.4. Análisis de errores	60
20.5. Discusión	60
21.Evaluación en PAWS-X — Benchmark Estándar	61
21.1. Motivación y descripción del dataset	61
21.2. Configuraciones evaluadas	61
21.3. Resultados	61
21.4. Análisis crítico	62
22.Comparativa: Groq Cloud vs. Ollama Local	64
22.1. Setup	64
22.2. Comparativa cuantitativa	64
22.3. Análisis costo / beneficio	64
23.Fine-tuning de BETO en PAWS-X (Google Colab)	66
23.1. Motivación	66
23.2. Justificación de la elección	66
23.3. Pipeline de entrenamiento	66
23.4. Diseño del pipeline local de evaluación	67
23.5. Resultados reales obtenidos	68
23.5.1. Evaluación en PAWS-X test	68
23.5.2. Evaluación en el dataset Exp.2 graduado	68
23.5.3. Hallazgo clave: comportamientos complementarios	68
23.6. Comparativa global actualizada	69
23.7. Reproducibilidad	70
IV Resultados, Análisis y Próximos Pasos	71
24.Resumen Cuantitativo de Experimentos y Contraste	71
24.1. Tabla maestra de experimentos	71
24.2. Contraste — qué nos dijo cada experimento	72
24.3. Contraste de errores — hallazgo central de la semana	72
24.4. Contraste contra la línea base de Luna (2022)	73

24.5. Matrices de confusión visuales	74
24.6. Trade-off Precision vs Recall	75
24.7. Análisis cualitativo — ejemplos de errores	75
24.7.1. Los dos falsos positivos de llama3.1:8b en Exp.2	75
24.7.2. Los tres falsos negativos de BETO en Exp.2	76
24.8. Análisis costo / beneficio	77
24.9. Lecciones aprendidas	77
25. Conclusiones Finales Actualizadas y Trabajo Siguiente	79
25.1. Resumen del avance	79
25.2. Decisiones técnicas validadas	79
26. Posibles Próximos Pasos (a Consultar con el Asesor)	80
26.1. Resultado clave que motiva los próximos pasos	80
26.2. Candidato 1 — Ensamble BETO + llama3.1:8b	80
26.3. Candidato 2 — Pipeline híbrido por umbrales	80
26.4. Candidato 3 — Filtrado de <i>front-matter</i>	81
26.5. Candidato 4 — Escalado de la evaluación en PAWS-X	81
26.6. Candidato 5 — Detección a nivel <i>span</i>	81
26.7. Candidato 6 — Detección a nivel documento (corpus completo)	81
26.8. Candidato 7 — Aplicación específica a Lic. Computación	81
26.9. Candidato 8 — Generar paráfrasis sintéticas a escala	82
26.10. Candidato 9 — Descarga de PDFs de CBS y CSH	82
26.11. Candidato 10 — Costos de inferencia detallados	82
26.12. Resumen visual de los candidatos	82
Anexo A. Estructura del Repositorio	83
Anexo B. Catálogo de Scripts	84
Anexo C. Contactos Adicionales	85
Anexo D. Comandos de Reproducción Esenciales	85

Índice de figuras

1. Distribución de resultados de extracción para la división CBI.	21
---	----

2.	Comparativa de F1 estimado por enfoque (valores estimados basados en literatura)	46
3.	Comparación de longitud: las paráfrasis son en promedio 28.6 % más largas.	52
4.	Similitud coseno por par. La línea roja marca el promedio global.	52
5.	Ejecución de <code>bash run_icl.sh</code> — clasificación de los 20 pares (CPU Intel i7-4770HQ, sin GPU, temperatura=0)	54
6.	Tabla de métricas del Experimento 1 — llama3.1:8b con 4 ejemplos ICL .	55
7.	Arquitectura del pipeline: extracción PDF → MongoDB → exportación y parafraseo	56
8.	Esquema completo del documento MongoDB por tesis	57
9.	Llama 3.1 8B obtiene F1 perfecto en negativos difíciles (misma tesis, inicio vs. final) y desciende a 0.857 global por falsos positivos en fragmentos formulaicos.	59
10.	PAWS-X español: MiniLM no resiste los negativos adversariales; el LLM con ICL mejora claramente la F1 y la precisión.	62
11.	BETO fine-tuneado en PAWS-X obtiene $F1 = 0.893$, situándose entre el LLM en cero-shot y la línea base de Luna (evaluada en su propio corpus, no en PAWS-X).	69
12.	Matrices de confusión en PAWS-X español. MiniLM marca casi todo como paráfrasis ($FP \approx TP$). Groq y BETO equilibran mucho mejor los cuatro cuadrantes, aunque BETO se beneficia del entrenamiento supervisado. . . .	74
13.	Matrices de confusión sobre el dataset propio Exp.2 (39 pares). Visualmente: llama3.1:8b llena la celda TP pero también tiene 2 FP. BETO no tiene ningún FP pero llena la celda FN (3 paráfrasis perdidas). Sus errores nunca caen en la misma celda: prueba gráfica del hallazgo de la sección anterior.	74
14.	Plano Precision–Recall con líneas de iso-F1 (curvas grises). Las dos configuraciones de MiniLM se sitúan en la esquina inferior derecha (alto recall, baja precisión). BETO en PAWS-X domina el cuadrante superior derecho. El par BETO/llama3.1:8b en el dataset propio se ubica en <i>esquinas opuestas</i> del plano — evidencia gráfica de su complementariedad.	75

Índice de tablas

1.	Stack tecnológico completo de la Semana 3.	11
2.	Lista completa de los 93 PDFs clasificados como vacíos en CBI.	26

3.	Modelos de embeddings para detección de paráfrasis	30
4.	Modelos NLI para clasificación cero-shot de paráfrasis	33
5.	LLMs generativos con soporte ICL few-shot (viabiles en 15 GB RAM con cuantización)	34
6.	Variantes GGUF de Qwen 2.5 7B Instruct y sus requerimientos de RAM .	34
7.	Variantes GGUF de Llama 3.1 8B Instruct	35
8.	Modelos base en español para fine-tuning	35
9.	Estadísticas del dataset PAWS-X en español	37
10.	Estadísticas del dataset STS-ES	38
11.	Hardware de la máquina de desarrollo	41
12.	Tabla comparativa completa de todos los enfoques evaluados	45
13.	Modelos evaluados y descartados con justificación	49
14.	Resultados de la exploración de parafraseo	51
15.	Tabla maestra de los 11 experimentos de la Semana 3.	71
16.	Catálogo de los 13 scripts Python desarrollados en la Semana 3.	84

Parte I

Construcción del Corpus

1 Introducción y Contexto del Proyecto

1.1 Objetivo general

Este documento consolida el trabajo realizado durante la **Semana 3** del Proyecto Terminal *Detección de Paráfrasis en Documentos Académicos*. El proyecto tiene como objetivo construir y evaluar un sistema de detección de paráfrasis aplicado al corpus de tesis de la Universidad Autónoma Metropolitana, Unidad Iztapalapa (UAM-I), extendiendo la tesis de maestría de **David Luna Luna (2022)** mediante la transición de modelos CNN+RNN hacia **Modelos de Lenguaje de Gran Escala (LLMs)** con *in-context learning*.

1.2 Antecedentes (Semanas 1 y 2)

En las semanas previas se estableció la base del proyecto:

- Análisis de la tesis base de David Luna (corpus CBI-M-00-21: 817 tesis, F1=0.970 con LSTM).
- Construcción del corpus TESIUAMI mediante *web scraping*: **21,286 registros** de tesis de las divisiones CBI, CBS y CSH.
- Descarga de **5,062 PDFs** de la división CBI (≈ 28 GB en disco).

1.3 Objetivos de la Semana 3

Las tareas asignadas por el Dr. Moreno Montiel para esta semana son:

1. **Base de datos no relacional:** Implementar una base de datos que almacene los PDFs referenciados junto con su texto plano extraído en formato JSON.
2. **Extracción de texto:** Aplicar el pipeline PDF $\rightarrow$ texto plano al corpus completo de CBI.
3. **Búsqueda de modelos LLM** orientados a paráfrasis en español.
4. **In-Context Learning:** Investigar la técnica para su implementación.

5. **Exploración con LLMs:** Experimentar con herramientas disponibles.

1.4 Stack tecnológico

Tecnología	Versión	Uso	Categoría
Python	3.12	Lenguaje principal de todos los scripts	Lenguaje
MongoDB	7.0	Base de datos no relacional del corpus	BD NoSQL
Docker	27+	Contenedor para MongoDB	Infra.
PyMuPDF	1.24+	Extracción de texto nativa de PDFs	Extracción
Tesseract OCR	5.x	OCR para PDFs escaneados	Extracción
pytesseract	0.3.x	Interfaz Python para Tesseract	Extracción
Pillow	10.x	Renderizado de páginas a imagen	Extracción
pymongo	4.12.1	Driver Python para MongoDB	BD
Ollama	latest	Servidor local de LLMs	LLM
llama3.1:8b	Q4_K_M	LLM para parafraseo y detección ICL	LLM
sentence-transformers	3.x	Embeddings para similitud coseno	NLP
paraphrase-multilingual-MiniLM-L12-v2	—	Modelo embeddings multilingüe	NLP
rich	13.x	Tablas y barras de progreso en terminal	UI

Tabla 1: Stack tecnológico completo de la Semana 3.

1.5 Estructura del trabajo

El trabajo de esta semana se desarrolló en dos etapas:

1. **Prueba piloto** (`prueba_piloto_extraccion/`): validación del pipeline sobre una muestra de 50 PDFs antes de escalar.
2. **Implementación masiva** (`extraccion_corpus_mongodb/`): base de datos MongoDB + extracción de los 5,060 PDFs de CBI + experimentos LLM.

2 Prueba Piloto de Extracción (50 PDFs)

2.1 Motivación

Antes de procesar los 5,062 PDFs del corpus completo, se realizó una **prueba piloto** con 50 PDFs para:

- Validar que PyMuPDF y Tesseract funcionaban correctamente con los PDFs de TESIAMI.
- Medir la distribución entre PDFs con texto nativo y PDFs escaneados.
- Estimar el tiempo de procesamiento y definir los parámetros óptimos.
- Detectar problemas antes de comprometer el tiempo del proceso masivo.

2.2 Lógica del pipeline de extracción

1. Abrir el PDF con `fitz.open()`.
2. Extraer el texto con `page.get_text()`.
3. Si ≥ 80 chars $\rightarrow$ **texto nativo** (ok/texto).
4. Si < 80 chars $\rightarrow$ renderizar a 300 DPI + Tesseract OCR.
5. Si OCR ≥ 80 chars $\rightarrow$ ok/ocr; si no $\rightarrow$ vacío.
6. Si `fitz.open()` falla $\rightarrow$ error.

2.3 Resultados de la prueba piloto

Estado	Cantidad	Porcentaje
Texto directo	45	90 %
OCR aplicado	0	0 %
Vacío	3	6 %
Error/corrupto	2	4 %
Total	50	100 %

Conclusión de la prueba piloto: El 90 % de los PDFs de CBI contienen texto digital extraíble directamente. El pipeline funciona correctamente y está listo para escalar a los 5,062 documentos.

3 Base de Datos No Relacional con MongoDB

3.1 Justificación de la elección tecnológica

Opción	Ventajas	Limitaciones	Decisión
TinyDB	Ya instalado, sin servidor	Lento >1,000 docs, sin índices	Descartado
Archivos .txt	Simplicidad	Sin consultas, miles de archivos	Descartado
MongoDB	Escalable, JSON nativo, índices	Requiere servidor	Elegido

3.2 Instalación con Docker

```

1 sudo docker run -d --name mongodb -p 27017:27017 \
2   --restart unless-stopped mongo:7

```

Listing 1: Despliegue de MongoDB con Docker

3.3 Esquema del documento MongoDB

```

1 {
2   "recno": "7", "doc": "UAM0007.PDF", "division": "CBI",
3   "titulacion": "DOCTORADO EN CIENCIAS (MATEMATICAS)",
4   "title": "ALGUNOS ASPECTOS DE RESOLUBILIDAD...",
5   "year": "2023",
6   "authors": ["LUIS MIGUEL VILLEGAS SILVA"],
7   "advisors": ["IVAN", "MIKHAIL"],
8   "pdf_path": "/corpus_tesiMiami/CBI/pdfs/UAM0007.PDF",
9   "pdf_disponible": true,
10  "extraccion": {
11    "estado": "ok", "metodo": "texto",
12    "texto_plano": "FUNDAMENTOS E IMPLICACIONES...",
13    "num_paginas": 89, "num_caracteres": 97387,
14    "tamano_bytes": 2974868, "error": null,
15    "fecha_procesado": "2026-05-19T20:55:00"
16  },
17  "fecha_carga": "2026-05-19T20:42:00"
18 }

```

Listing 2: Esquema de un documento en `tesiMiami.tesis`

3.4 Índices creados

Campo(s)	Tipo	Propósito
recno	Único	Prevenir duplicados; clave de upsert
division	Simple	Filtrar por división académica
(division, extraccion.estado)	Compuesto	Cola de trabajo resumible
pdf_disponible	Simple	Filtrar PDFs disponibles

4 Scripts e Implementaciones

4.1 config.py — Configuración central

```

1 from pathlib import Path
2
3 PROJECT_ROOT = Path("/home/purple-alien/Escritorio/Proyecto terminal 1")
4 CORPUS_ROOT = PROJECT_ROOT / "corpus_tesiunami"
5 INDICE_GLOBAL = CORPUS_ROOT / "indice_global.json"
6
7 MONGO_URI = "mongodb://localhost:27017"
8 DB_NAME = "tesiunami"
9 COLLECTION_NAME = "tesis"
10
11 MIN_TEXT_CHARS = 100 # umbral texto nativo vs OCR
12 OCR_DPI = 300 # resolucion de renderizado
13 OCR_LANG = "spa" # idioma Tesseract
14 DIVISION_ACTIVA = "CBI"

```

Listing 3: config.py — Parámetros centralizados

4.2 01_cargar_indice.py — Carga inicial de la base de datos

Lee el `indice_global.json` (21,286 registros) e inserta documentos en MongoDB de forma idempotente con `$setOnInsert`: si el `recno` ya existe, no lo sobrescribe.

```

1 ops.append(UpdateOne(
2     {"recno": doc["recno"]}, # clave unica
3     {"$setOnInsert": doc}, # solo escribe si no existe
4     upsert=True
5 ))
6 if len(ops) >= 500:
7     col.bulk_write(ops, ordered=False)
8     ops = []

```

Listing 4: Inserción idempotente por lotes con `bulk_write`

Resultado: 21,284 registros insertados. CBI: 5,120 registros, de los cuales 5,060 tienen PDF en disco.

4.3 02_extraer_corpus.py — Pipeline masivo de extracción

Script central de la Semana 3. Consulta MongoDB por registros `estado="pendiente"` y extrae el texto con fallback a OCR. Diseño **reanudable**: al interrumpirse, retoma desde donde quedó.

```

1 def extraer_texto(pdf_path: Path) -> dict:
2     try:
3         doc = fitz.open(str(pdf_path))
4         resultado["num_paginas"] = len(doc) # CRITICO: dentro del try
5         texto = "".join(page.get_text() for page in doc)
6         doc.close()
7     except Exception as e:
8         resultado.update({"estado": "error", "error": str(e)})
9         return resultado
10
11     if len(texto.strip()) >= MIN_TEXT_CHARS:
12         resultado.update({"estado": "ok", "metodo": "texto", "texto_plano": texto})
13         return resultado
14
15     # Fallback: OCR con Tesseract a 300 DPI
16     try:
17         doc = fitz.open(str(pdf_path))
18         paginas = []
19         for page in doc:
20             mat = fitz.Matrix(OCR_DPI / 72, OCR_DPI / 72)
21             pix = page.get_pixmap(matrix=mat, colorspace=fitz.csRGB)
22             img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
23             paginas.append(pyesseract.image_to_string(img, lang=OCR_LANG))
24         doc.close()
25         texto_ocr = "\n".join(paginas)
26         estado = "ok" if len(texto_ocr.strip()) >= MIN_TEXT_CHARS else "vacio"
27         resultado.update({"estado": estado, "metodo": "ocr", "texto_plano": texto_ocr})
28     except Exception as e:
29         resultado.update({"estado": "error", "error": f"OCR: {e}"})
30     return resultado

```

Listing 5: Función de extracción con fallback a OCR

4.3.1 Bug crítico corregido durante la ejecución

Bug: RuntimeError en PDF #2,881 — len(doc) fuera del try

Error: RuntimeError: code=7: Invalid number of pages

Causa: resultado["num_paginas"] = len(doc) estaba ubicado *fuera* del bloque try, por lo que PDFs con conteo de páginas corrupto lanzaban una excepción no capturada, deteniendo todo el proceso.

Solución: mover len(doc) *dentro* del mismo try/except. El diseño resumible de MongoDB permitió reanudar exactamente desde el PDF #2,881 sin reprocesar los 2,880 anteriores.

4.4 03_stats_bd.py — Estadísticas y monitoreo

Genera tablas de resumen en terminal con la librería rich, mostrando distribución por estado de extracción, método y errores recientes.

```
1 for estado in ["pendiente", "ok", "vacio", "error"]:
2     count = col.count_documents({
3         "division": "CBI", "extraccion.estado": estado
4     })
5     print(f"{estado}: {count}")
```

Listing 6: Consulta de estadísticas por estado

4.5 04_exportar_json.py — Exportación del corpus a JSON

Genera dos archivos JSON desde MongoDB para uso externo:

```
1 def doc_a_dict(doc, incluir_texto=True) -> dict:
2     ext = doc.get("extraccion", {})
3     return {
4         "recno": doc.get("recno"),    "doc": doc.get("doc"),
5         "titulo": doc.get("title"),  "anio": doc.get("year"),
6         "autores": doc.get("authors", []),
7         "titulacion": doc.get("titulacion"),
8         "division": doc.get("division"),
9         "texto_plano": {
10             "estado": ext.get("estado"),    "metodo": ext.get("metodo"),
11             "num_paginas": ext.get("num_paginas"),
12             "num_caracteres": ext.get("num_caracteres"),
13             "contenido": ext.get("texto_plano") if incluir_texto else None,
```

```

14     },
15 }

```

Listing 7: Estructura del documento JSON exportado

Archivo	Contenido	Registros	Tamaño
corpus_cbi_muestra.json	Metadatos + texto plano	79	10.4 MB
corpus_cbi_metadata.json	Solo metadatos	5,120	3.8 MB

4.6 05_explorar_parafrasis.py — Exploración de parafraseo con LLM

Selecciona 6 tesis del corpus CBI (una por área temática), extrae un fragmento de 600 caracteres, lo envía a llama3.1:8b con un prompt especializado y calcula la similitud coseno entre original y paráfrasis.

```

1 def construir_prompt(texto, titulacion):
2     return f"""Eres un asistente academico experto en reescritura de texto
3     cientifico en espanol. Parafrasea el siguiente fragmento de una tesis
4     universitaria ({titulacion}) manteniendo el mismo significado.
5
6     Reglas:
7     - Mantén el significado sin agregar ni quitar informacion.
8     - Usa sinonimos y reestructura las oraciones.
9     - Conserva los terminos tecnicos especializados.
10    - El resultado debe tener longitud similar al original.
11    - Responde UNICAMENTE con el texto parafraseado.
12
13    TEXTO ORIGINAL: {texto}
14    PARAFRASIS: """

```

Listing 8: Prompt de parafraseo académico científico

4.7 06_deteccion_icl.py — Detección ICL (Experimento 1)

Implementa la detección de paráfrasis con *In-Context Learning*: construye un dataset de 20 pares (6 positivos + 14 negativos) y clasifica cada par con llama3.1:8b usando 4 ejemplos few-shot fijos.

```

1 def construir_prompt(texto_a, texto_b):
2     return f"""Eres un experto en deteccion de parafrasis academicas.

```

```

3  Responde UNICAMENTE con: SI o NO.
4
5  Ejemplo 1:
6  TEXTO A: El control predictivo minimiza una funcion objetivo con restricciones.
7  TEXTO B: Se busca minimizar un criterio de desempeno sujeto a limitaciones.
8  Son parafrasis? -> SI
9
10 Ejemplo 2:
11 TEXTO A: Los monocristales de Silicio se usan en electronica.
12 TEXTO B: El reactor opera a 300 K para mantener la reaccion optima.
13 Son parafrasis? -> NO
14
15 [...2 ejemplos mas...]
16
17 TEXTO A: {texto_a[:400]}
18 TEXTO B: {texto_b[:400]}
19 Son parafrasis? ->""
20
21 # Temperatura=0 para respuesta determinista
22 respuesta = ollama.chat(model="llama3.1:8b",
23     messages=[{"role": "user", "content": prompt}],
24     options={"temperature": 0.0, "num_predict": 10})

```

Listing 9: Prompt few-shot para detección de paráfrasis

4.8 07_icl_dificil.py — Dataset difícil (Experimento 2)

Construye un dataset de dificultad graduada con tres niveles de negativos:

Nivel	Descripción	Ejemplo
Positivos	(fragmento, paráfrasis generada)	Original vs. llama3.1:8b
Fáciles (NO)	Áreas completamente distintas	Matemáticas vs. Ing. Química
Medios (NO)	Misma área, tesis distintas	Dos tesis de Física
Difíciles (NO)	Misma tesis, sección diferente	Introducción vs. Conclusiones

5 Secuencia de Ejecución

```
1 cd extraccion_corpus_mongodb/
2
3 # PASO 1: cargar 21,284 registros en MongoDB (~20 segundos, idempotente)
4 python3.12 01_cargar_indice.py
5
6 # PASO 2: extraer texto de 5,060 PDFs CBI (~6 horas, reanudable)
7 nohup python3.12 02_extraer_corpus.py >> extraccion.log 2>&1 &
8
9 # PASO 3: estadísticas finales
10 python3.12 03_stats_bd.py
11
12 # PASO 4: exportar JSON (muestra + metadatos)
13 bash run_icl.sh 04_exportar_json.py
14
15 # PASO 5: explorar parafraseo con llama3.1:8b (~30 min, 6 tesis)
16 bash run_icl.sh 05_explorar_parafrasis.py
17
18 # PASO 6: experimento 1 ICL (~20 min, 20 pares)
19 bash run_icl.sh 06_deteccion_icl.py
20
21 # PASO 7: experimento 2 dataset difícil
22 bash run_icl.sh 07_icl_dificil.py
```

Listing 10: Pipeline completo de Semana 3

Ejecución de scripts LLM

Los scripts 05–07 requieren el venv de `prueba_piloto_extraccion/.venv` que tiene instalado `ollama`, `sentence-transformers` y `pymongo`. El script auxiliar `run_icl.sh` activa el venv correcto automáticamente.

6 Resultados Finales de la Extracción CBI

6.1 Resumen ejecutivo

La extracción de los 5,060 PDFs de CBI se completó al **100 %**:

Resultado	Documentos	Porcentaje
ok — texto nativo (PyMuPDF)	4,953	97.9 %
ok — OCR (Tesseract)	1	0.02 %
vacío (sin páginas)	93	1.8 %
error (corrupto)	13	0.3 %
Total procesados	5,060	100 %

Resultados de extracción — División CBI (5,060 PDFs)

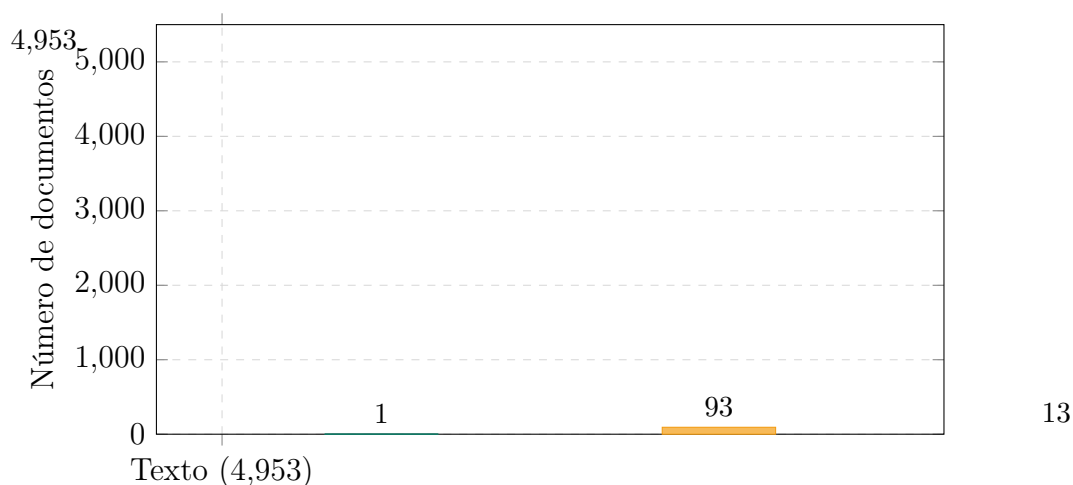


Figura 1: Distribución de resultados de extracción para la división CBI.

6.2 Estadísticas del texto extraído

Estadística	Valor
Mínimo de caracteres	124
Máximo de caracteres	2,909,338
Media de caracteres	129,249
Mediana de caracteres	110,572
Total de páginas procesadas	≈350,000
Tiempo total de extracción	≈6 horas

7 Análisis de PDFs Problemáticos

7.1 Tipo A — Árbol de páginas corrupto (5 PDFs)

Archivo	Año	Título / Error
UAMI12320.pdf	2005	SIMULACION Y OPTIMIZACION TECNICA DE UN SISTEMA DE COGENERACION <i>code=7: cannot find page 3 in page tree</i>
UAMI12334.PDF	1995	ELABORACION DE MICROENCAPSULADOS FUNCIONALES DE CAMARON
UAMI12468.pdf	1990	SISTEMA DE MONITOREO — <i>code=7: cannot find page 3</i>
UAMI13152.pdf	2006	PROPUESTA DE UN CRITERIO DE SIMILITUD PARA TRANSFORMACION
206215613.PDF	—	MATERIAL XML EN APOYO A LENGUAJES DE PROGRAMACION

7.2 Tipo B — Número de páginas inválido (5 PDFs)

Archivo	Año	Título
UAMI16741.pdf	2016	DELIMITACION Y CARACTERIZACION DE CUENCAS HIDROLOGICAS
203213044.PDF	—	PERSONAL SOFTWARE PROCESS Y TEAM SOFTWARE PROCESS
204244658.PDF	2013	APLICACION PARA EL MANEJO DE ECOSISTEMAS LITORALES
204323795.PDF	2011	SISTEMA PUNTO DE VENTA AL DETALLE
205317721.PDF	2008	DESARROLLO DE SISTEMAS UTILIZANDO PSP

7.3 Tipo C — Error de capitalización (3 PDFs recuperables)

Archivos que *existen* en disco con extensión `.pdf` minúscula, pero el índice los registró con `.PDF` mayúscula. En Linux el sistema de archivos es *case-sensitive*.

Ruta en BD	Archivo real	Título
UAMI13886.PDF	UAMI13886.pc	DOCUMENTACION Y PROTOCOLOS DE REVISION
UAMI17578.PDF	UAMI17578.pc	DICCIONARIO LSM-ESPAÑOL Y ESPAÑOL-LSM
UAMI17670.PDF	UAMI17670.pc	ESTUDIO DE PARAMETROS DE CALIDAD Y EFICIENCIA

Corrección disponible

El script `fix_capitalizacion.py` crea enlaces simbólicos y actualiza el estado en MongoDB para re-extracción automática: `python3.12 fix_capitalizacion.py`

7.4 PDFs vacíos (93 documentos)

Todos presentan `num_paginas = 0` según PyMuPDF. La mayoría son documentos anteriores a 1995 generados con herramientas de conversión PDF no estandarizadas.

Archivo	Año	Págs.	Título
UAM0581.PDF	2020	0	ANALISIS DE ESPECTROS DE ABSORCION Y DISPERSION
UAM0584.PDF	2000	0	ESTABILIZACION DE SISTEMAS AFINES
UAM1586.PDF	1998	0	RIESGO Y SEGURIDAD AMBIENTAL
UAM1787.PDF	1992	0	PROYECTO DE PRE-REGISTRO A CURSOS
UAM1954.PDF	1988	0	PROGRAMA PARA DETERMINAR PRIORIDAD
UAM1976.PDF	1998	0	UNIDAD PARA DISMINUIR COMPUESTOS PRECURSORES
UAM3354.PDF	1995	0	SISTEMA INTEGRAL GENERICO CONTABLE
UAM3564.PDF	2004	0	APLICACION A TELEVISION POR CABLE
UAM3584.PDF	1992	0	REDES NEURONALES: ALGORITMO DE RETROPAGACION
UAM3598.PDF	1997	0	POLITICAS DE MANTENIMIENTO
UAM3599.PDF	2015	0	ANALISIS DE REMODELACION DEL AREA DE CUNEROS
UAM3636.PDF	1995	0	MICROENCAPSULADO DE EMULSION MULTIPLE
UAM6819.PDF	—	0	AHORRO DE ENERGIA EN VAPOR DE ALTA PRESION
UAM7255.PDF	2000	0	INVENTARIOS
UAM7263.PDF	2016	0	REDUCCION DE MODELOS DE COLUMNAS DE DESTILACION

Archivo	Año	Págs.	Título
UAM7389.PDF	2004	0	CURSO DE ESTRUCTURA DE DATOS
UAM7582.PDF	1995	0	RESINAS DE INTERCAMBIO IONICO
UAM7685.PDF	2023	0	EL PROBLEMA INVERSO DEL CALCULO DE VARIACIONES
UAM7699.PDF	2015	0	SISTEMA TIPICO DE COMPRESION-VAPOR
UAM7914.PDF	2003	0	BANCA INTEGRAL POR TERMINAL
UAM7975.PDF	1991	0	PRODUCCION DE SABORES DE QUESO
UAM8007.PDF	1996	0	SISTEMA BASADO EN MICROPROCESADOR 8031
UAM8008.PDF	2017	0	EQUIPO PARA DIAGNOSTICO MEDICO CON ULTRASONIDO
UAM8106.PDF	2023	0	RECONSTRUCCION DE PATRONES BINARIOS
UAM8648.PDF	1988	0	INSTALACIONES ELECTRICAS PARA HOSPI-TALES
UAMI10315.PDF	2000	0	MONITOREO FETAL ELECTRONICO
UAMI10348.PDF	2003	0	SISTEMA INTEGRAL ADMINISTRATIVO DE CONTABILIDAD
UAMI10349.PDF	2003	0	CODIFICACION DICOM
UAMI10360.PDF	2015	0	PLANEACION DE RED LAN
UAMI10361.PDF	2003	0	REDISEÑO DE PLANTA PRODUCTORA DE DIESEL
UAMI10367.PDF	2002	0	EFEECTO COARTICULATORIO EN ESPAÑOL
UAMI10375.PDF	2004	0	CIBAC: TRATAMIENTO DE AGUAS RESIDUA-LES
UAMI10386.PDF	2003	0	OPTIMIZACION EN COMPUTO DE ALTO REN-DIMIENTO
UAMI10394.PDF	1993	0	PLANTA PRODUCTORA DE ENDULCORANTE NATURAL
UAMI10416.PDF	2017	0	ANALISIS HISTORICO DEL CLIMA EN MEXICO
UAMI10457.PDF	2000	0	INTRANET CALL CENTER
UAMI10459.PDF	1999	0	APOYO A FUENTES BIBLIOGRAFICAS
UAMI10460.PDF	2016	0	ANALISIS DE ESTABILIDAD EN BWR'S
UAMI10611.PDF	2003	0	UNIDAD DE MONITOREO
UAMI10644.PDF	1990	0	EQUIPAMIENTO DE CENTRO DE REHABILI-TACION
UAMI10649.PDF	1989	0	TRATAMIENTO DE AGUAS COCA-COLA
UAMI10654.PDF	2000	0	EDUCACION A DISTANCIA
UAMI10697.PDF	2007	0	ATHAPASCAN-1: BALANCE DE CARGA
UAMI10712.PDF	2013	0	REDES NEURONALES EN RECONOCIMIENTO
UAMI10726.PDF	2006	0	PATRON DE VIENTO EN MEXICO
UAMI10730.PDF	2003	0	CONTROLADORES DIFUSOS CON ALGORIT-MOS GENETICOS

Archivo	Año	Págs.	Título
UAMI10758.PDF	2023	0	HERRAMIENTA DE ASIGNACION DINAMICA
UAMI10805.PDF	1995	0	MECANISMOS DE SINCRONIZACION
UAMI10809.PDF	1995	0	SISTEMA ORIENTADO A OBJETOS
UAMI10813.PDF	2003	0	INSTRUMENTO VIRTUAL DE CONTROL
UAMI10834.PDF	1989	0	EXTRACCION DE ORO POR LIXIVIACION
UAMI10855.PDF	1993	0	REGISTRO DE PROPUESTAS
UAMI10856.PDF	2003	0	PROTOCOLOS PARA MEDICION DE GASTO CARDIACO
UAMI10860.PDF	2004	0	BASE DE DATOS PARA HISTORIA CLINICA
UAMI10862.PDF	1997	0	UNIDAD DE ELECTROCIRUGIA
UAMI11841.PDF	2013	0	SECADOR SOLAR POR CONVECCION NATURAL
UAMI12762.pdf	2012	0	APLICACIONES DE SINTESIS DE TEXTO A VOZ
UAMI12880.pdf	2014	0	SIMULACION DE TRANSITO EN LAGO DE CHAPALA
UAMI12915.pdf	2006	0	EVALUACION DE MICROSCOPIOS QUIRURGICOS
UAMI13070.pdf	2024	0	APLICACION WEB DE STREAMING
UAMI13167.pdf	2007	0	INFRAESTRUCTURA DE MEDICINA NUCLEAR
UAMI13207.pdf	2001	0	SIMULADOR CENTRAL TERMoeLECTRICA VALLE DE MEXICO
UAMI14728.pdf	2015	0	SIMULACION PARALELA DE PROCESOS DE INTRUSION
UAMI15893.pdf	2005	0	DESCONTAMINACION RADIOLOGICA EN HUMANOS
UAMI15897.pdf	1990	0	SISTEMA DE ATENCION DE SERVICIOS
UAMI16216.pdf	1987	0	MODELANDO A MITTAG-LEFFLER
UAMI16553.pdf	2001	0	FUNCIONALIZACION DE SUSTRATOS NANOESTRUCTURADOS
UAMI16759.pdf	2015	0	MAPEO DE PROCESOS EN HEMODIALISIS
UAMI16766.pdf	1988	0	SIMULACION DE REDES POROSAS EN COMPUTO PARALELO
UAMI17165.pdf	2025	0	VALIDACION PARA GARANTIZAR CALIDAD DE SISTEMAS
UAMI17458.pdf	2016	0	DEGRADACION FOTOSONOCATALITICA
UAMI17539.pdf	2016	0	SANEAMIENTO DE AGUAS RESIDUALES
UAMI17664.pdf	2015	0	INTEGRACION TERMICA Y OPTIMIZACION
UAMI17761.pdf	2001	0	SINTESIS DE 1,2,3-TRIAZOLES
UAMII22985.pdf	1987	1	DISPOSITIVO ANALITICO BASADO EN NANO-TECNOLOGIA (OCR: 70 chars)

Archivo	Año	Págs.	Título
UAM20661.PDF	1987	0	MECANICA DEL SISTEMA VASCULAR
UAM21042.PDF	1996	0	VECTOCARDIOGRAFIA EN TRES DIMENSIONES
UAM21295.PDF	1986	0	MEDICINA NUCLEAR: OBTENCION DE IMAGENES
UAM21296.PDF	1987	0	MECANICA RESPIRATORIA
UAM21299.PDF	1984	0	APARATO GASTROINTESTINAL
UAM21320.PDF	1999	0	MADUREZ PULMONAR FETAL
UAM21439.PDF	1992	0	RECONOCIMIENTO DE PATRONES MEDIANTE REDES NEURONALES
UAM21581.PDF	1993	0	SISTEMA DE CONTROL ESCOLAR
UAMI22403.pdf	2008	0	SENSOR POR IMPEDANCIA ELECTRICA
UAMI22481.pdf	2023	0	PROPIEDADES ESTRUCTURALES Y TERMODINAMICAS
201321493.PDF	1990	0	SISTEMA PARA CONTROL DE CALIDAD DEL SERVICIO
203213743.PDF	2012	0	PERSONAL SOFTWARE PROCESS & TEAM SOFTWARE PROCESS
204214001.PDF	1989	0	TUTORIA DE OPTIMIZACION BASADO EN AGENTE VIAJERO
204215845.PDF	2011	0	METODOLOGIA INTEGRAL PARA EVALUAR RENDIMIENTO
205319454.PDF	2015	0	DESARROLLO DE SISTEMAS DE SOFTWARE CON PSP
206320468.PDF	2023	0	HEAT TRANSFER PROCESS IN PEBBLE FUEL
206321040.PDF	2012	0	PERSONAL SOFTWARE PROCESS & TEAM SOFTWARE PROCESS
206323123.PDF	2011	0	EVALUACION DE ALTERNATIVAS DE AHORRO DE ENERGIA

Tabla 2: Lista completa de los 93 PDFs clasificados como vacíos en CBI.

Parte II

Investigación de Modelos y Estado del Arte

8 Fundamentos Teóricos: Aprendizaje en Contexto (ICL)

8.1 Definición y Origen

El Aprendizaje en Contexto (*In-Context Learning*, ICL) fue formalizado por Brown et al. (2020) con GPT-3. A diferencia del *fine-tuning* (que modifica los pesos del modelo mediante retropropagación), el ICL proporciona ejemplos etiquetados directamente dentro del texto de entrada (*prompt*), permitiendo que el modelo infiera el patrón de la tarea sin ningún ajuste de parámetros.

Formalmente, dado un modelo M y una tarea T , el ICL con k ejemplos (*few-shot*) es:

$$y^* = \arg \max_y P_M(y \mid x^{(1)}, y^{(1)}, \dots, x^{(k)}, y^{(k)}, x_{\text{test}}) \quad (1)$$

donde $(x^{(i)}, y^{(i)})$ son los pares de ejemplos de entrenamiento incluidos en el contexto y x_{test} es la instancia a clasificar.

8.2 Modalidades de ICL

Zero-shot El modelo recibe únicamente la instrucción de la tarea y el ejemplo a clasificar, sin ningún ejemplo etiquetado. Depende totalmente del conocimiento preentrenado.

One-shot Se incluye un único ejemplo etiquetado. Mejora significativamente el rendimiento en tareas ambiguas.

Few-shot Se incluyen k ejemplos (k generalmente entre 2 y 16). Es la modalidad más estudiada y robusta.

Chain-of-Thought (CoT) Los ejemplos incluyen el razonamiento paso a paso antes de la respuesta. Demostrado por PlagBench (2025) que supera significativamente al *few-shot* simple en detección de plagio.

8.3 ICL aplicado a Detección de Paráfrasis

El benchmark PARAPHRASUS (Michail et al., COLING 2025) evaluó tres tipos de prompts para detección de paráfrasis con Llama 3 8B Instruct:

- **P1 (Directo):** “¿Son estas dos oraciones paráfrasis entre sí? Responde PARÁFRASIS o NO PARÁFRASIS.”
- **P2 (Semántico):** Instrucción explicitando que debe comparar el significado, no las palabras exactas.
- **P3 (Basado en recuperación):** Recupera ejemplos similares del corpus de ejemplos.

El hallazgo principal: **P1 con 4 ejemplos fue el mejor en promedio**, superando a prompts más elaborados. Esto sugiere que la simplicidad en el diseño del *prompt* favorece la generalización.

El formato recomendado para este proyecto, adaptado al español académico, es:

```

1 prompt = """Determina si las siguientes dos oraciones académicas expresan
2 el mismo significado (son paráfrasis) o no.
3
4 --- Ejemplo 1 ---
5 Oración A: "El algoritmo de clasificación alcanzó una precisión del 95%"
6 Oración B: "Se obtuvo un 95% de precisión con el clasificador implementado"
7 Respuesta: PARÁFRASIS
8
9 --- Ejemplo 2 ---
10 Oración A: "Se utilizó una red neuronal convolucional para el análisis"
11 Oración B: "El experimento se realizó en un entorno controlado de laboratorio"
12 Respuesta: NO PARÁFRASIS
13
14 --- Ejemplo 3 ---
15 Oración A: "Los resultados demuestran una mejora significativa del rendimiento"
16 Oración B: "Se observó una mejora estadísticamente significativa en el desempeño"
17 Respuesta: PARÁFRASIS
18
19 --- Ejemplo 4 ---
20 Oración A: "La metodología propuesta reduce el tiempo de cómputo"
21 Oración B: "El enfoque presentado disminuye el coste computacional"
22 Respuesta: PARÁFRASIS
23
24 --- A evaluar ---
25 Oración A: "{oracion_a}"

```

```
26 Oración B: "{oracion_b}"  
27 Respuesta: ""
```

Listing 11: Plantilla de prompt ICL para detección de paráfrasis en español académico

9 Catálogo Exhaustivo de Modelos en Hugging Face

Fuente de datos

Este catálogo fue construido mediante consulta directa a los *model cards* de Hugging Face (<https://huggingface.co>) en mayo de 2026. Los datos de descargas mensuales corresponden a la fecha de consulta.

Los modelos se organizan en cuatro categorías según su mecanismo de operación y adecuación al proyecto.

9.1 Categoría 1: Modelos de Embeddings para Similitud Semántica

Estos modelos convierten oraciones en vectores de alta dimensión (*embeddings*). La detección de paráfrasis se realiza calculando la similitud coseno entre los vectores de dos oraciones: si la similitud supera un umbral τ (típicamente 0.80–0.90), las oraciones se clasifican como paráfrasis.

Ventaja clave: no requieren GPU ni datos etiquetados; la inferencia es inmediata.

Limitación: la precisión depende fuertemente de la calibración del umbral τ .

Tabla 3: Modelos de embeddings para detección de paráfrasis

Modelo (ID en HF)	Params	Desc./mes	Dim.	Max tok.	Idiomas
sentence-transformers/ paraphrase-multilingual- MiniLM-L12-v2	118M	48.6M	384	128	50 idiomas
sentence-transformers/ paraphrase-multilingual- mpnet-base-v2	278M	7.1M	768	128	50 idiomas
hiiamsid/sentence_ similarity_spanish_es	110M	54,867	768	512	Español
somosnlp-hackathon-2022/ paraphrase-spanish- distilroberta	125M	3,941	768	128	ES + EN
Alibaba-NLP/ gte-multilingual-base	305M	1.02M	768	8,192	70+ idiomas

(Continuación)					
Modelo	Params	Desc./mes	Dim.	Max tok.	Idiomas
intfloat/ multilingual-e5-large-instr	560M	1.33M	1,024	512	94+ idiomas
mrm8488/multilingual-e5-large- ft-sts-spanish-matryoshka-	560M	329	1,024	512	ES+multi

9.1.1 Análisis Detallado de los Modelos de Embeddings más Relevantes

1. paraphrase-multilingual-MiniLM-L12-v2 (48.6 millones de descargas/mes) Es el modelo más descargado de toda la categoría y uno de los más descargados en Hugging-Face globalmente. Fue entrenado por Reimers y Gurevych (2019) mediante redes siamesas BERT para detección de paráfrasis en 50 idiomas. Convierte oraciones en vectores de 384 dimensiones.

- **Ventajas:** extremadamente rápido (~ 10 ms/par en CPU), ~ 250 MB en RAM, cubre español.
- **Limitaciones:** trunca a 128 tokens (problemático para párrafos académicos largos); las 384 dimensiones son menores que modelos de 768 dims.
- **Uso recomendado:** pre-filtrado rápido de pares candidatos a paráfrasis.

2. hiiamsid/sentence_similarity_spanish_es (54,867 descargas/mes) Basado en BETO (dccuchile/bert-base-spanish-wwm-cased) con *Mean Pooling*. Entrenado en el benchmark STS en español (sts_multi_mt). Es el único modelo de esta categoría **monolingüe en español** con una ventana de **512 tokens**, lo que lo hace especialmente adecuado para oraciones o párrafos académicos extensos.

- **Evaluación:** Spearman = 0.8233 en STS español.
- **Ventajas:** 512 tokens, monolingüe español, ~ 440 MB RAM.
- **Limitaciones:** solo español; no soporta seguimiento de instrucciones.

3. paraphrase-spanish-distilroberta (somosnlp-hackathon-2022) El modelo con el nombre más explícitamente orientado a paráfrasis en español. Fue creado durante el hackathon SomosNLP 2022 mediante destilación conocimiento del modelo inglés `paraphrase-mpnet-base-v1` hacia BERTIN (RoBERTa en español). Se entrenó en 8 corpus paralelos ES-EN.

- **Evaluación:** Pearson = 0.8495, Spearman = 0.8579 en STS español (el mejor Spearman entre modelos específicos de español).
- **Limitaciones:** ventana de solo 128 tokens; puede truncar textos académicos.

4. Alibaba-NLP/gte-multilingual-base (1.02 millones de descargas/mes) Arquitectura estado del arte con una característica crítica para texto académico: ventana de **8,192 tokens**, comparada con los 128–512 de los modelos anteriores. Soporta 70+ idiomas y produce embeddings de 768 dimensiones. Requiere `trust_remote_code=True` en Transformers.

- **Ventajas:** maneja párrafos completos sin truncamiento; estado del arte en recuperación multilingüe.
- **Limitaciones:** ~600 MB RAM; más lento que MiniLM.
- **Uso recomendado:** comparación de fragmentos extensos de tesis (múltiples párrafos).

5. mrm8488/multilingual-e5-large-ft-sts-spanish-matryoshka-768 (329 descargas/mes) El modelo con la evaluación más alta reportada: Spearman = **0.8775** en el conjunto de prueba de STS español. Está basado en `multilingual-e5-large` (XLM-RoBERTa-large) con fine-tuning en pares STS en español aumentados. Usa la técnica *Matryoshka Representation Learning*: un único modelo entrenado para producir embeddings útiles en múltiples dimensiones (768, 512, 256, 128, 64, 32, 16), permitiendo un intercambio entre precisión y eficiencia.

- **Limitaciones:** solo 329 descargas/mes (baja adopción; menor confiabilidad en producción); ~2.2 GB RAM.

9.2 Categoría 2: Modelos NLI para Clasificación Cero-Shot

La *Inferencia de Lenguaje Natural* (NLI) clasifica si una hipótesis se deriva de una premisa: *entailment* (se deriva), *neutral* o *contradiction* (contradice). Esta formulación puede adaptarse a detección de paráfrasis: si la hipótesis “esta oración tiene el mismo significado” obtiene alta probabilidad de *entailment*, las oraciones son paráfrasis.

Tabla 4: Modelos NLI para clasificación cero-shot de paráfrasis

Modelo	Params	Desc./mes	Max tok.	Idiomas
joeddav/xlm-roberta-large-xnli	560M	99,567	512	15 (incl. ES)
cross-encoder/nli-deberta-v3-small	100M	333,634	512	Solo EN
cross-encoder/nli-MiniLM2-L6-H768	82M	226,348	512	Solo EN

joeddav/xlm-roberta-large-xnli (**el único viable para español**) Fine-tuned de XLM-RoBERTa-large en el corpus XNLI (15 idiomas, incluyendo español). Permite clasificación cero-shot mediante la siguiente plantilla:

```

1 from transformers import pipeline
2
3 nli = pipeline("zero-shot-classification",
4               model="joeddav/xlm-roberta-large-xnli")
5
6 oracion_a = "El algoritmo alcanzó 95% de precisión"
7 oracion_b = "Se obtuvo 95% de precisión con el clasificador"
8
9 resultado = nli(
10     sequences=f"Oración A: {oracion_a}. Oración B: {oracion_b}.",
11     candidate_labels=["paráfrasis", "no es paráfrasis"],
12     hypothesis_template="Las dos oraciones son {}."
13 )
14 print(resultado)
15 # {'labels': ['paráfrasis', 'no es paráfrasis'],
16 #  'scores': [0.923, 0.077]}
```

Listing 12: Uso de NLI para detección cero-shot de paráfrasis

- **Ventajas:** cero-shot, sin datos etiquetados, sin entrenamiento.
- **Limitaciones:** 560M parámetros (~ 2.2 GB RAM); el español no es el idioma primario; más lento que los modelos de embeddings.
- **Nota:** los modelos cross-encoder/nli-* son solo en inglés y **no son utilizables** directamente para este proyecto.

9.3 Categoría 3: LLMs Generativos con ICL Verdadero

Estos son los modelos que implementan ICL en su forma más pura: se les proporciona una instrucción y k ejemplos etiquetados dentro del *prompt*, y generan texto que incluye la

clasificación. Requieren cuantización para ejecutarse en CPU con 15 GB de RAM.

Tabla 5: LLMs generativos con soporte ICL few-shot (viabiles en 15 GB RAM con cuantización)

Modelo	Params	Desc./mes	Ctx.	RAM (Q4)	Español
Qwen/Qwen2.5-7B-Instruct	7.6B	13.1M	128K	~7–8 GB	Sí (29 idiomas)
meta-llama/Meta-Llama-3.1-8B-Instruct	8B	11.0M	128K	~7–8 GB	Oficial

9.3.1 Qwen 2.5 7B Instruct

Desarrollado por Alibaba Cloud (Qwen Team, 2024). Arquitectura de transformador causal con 7.61B parámetros (6.53B no-embedding), *Grouped Query Attention* (GQA con 4 cabezas KV), ventana de contexto de **128,000 tokens**. Soporta 29 idiomas incluyendo español.

Tabla 6: Variantes GGUF de Qwen 2.5 7B Instruct y sus requerimientos de RAM

Cuantización	Tamaño archivo	RAM estimada	Calidad
Q2_K	3.02 GB	~5–6 GB	Baja
Q3_K_M	3.81 GB	~6–7 GB	Media-baja
Q4_K_M	4.68 GB	~7–8 GB	Recomendada
Q5_K_M	5.44 GB	~8–9 GB	Alta
Q6_K	6.25 GB	~9–10 GB	Muy alta
Q8_0	8.10 GB	~11–12 GB	Casi exacta

9.3.2 Meta Llama 3.1 8B Instruct

Desarrollado por Meta AI (2024). Arquitectura transformador autorregresivo con GQA, 8 mil millones de parámetros. El español es uno de los **8 idiomas oficialmente soportados** (inglés, alemán, francés, italiano, portugués, hindi, español, tailandés). Ventana de contexto de 128,000 tokens.

Acceso restringido

meta-llama/Meta-Llama-3.1-8B-Instruct requiere aceptar la *Llama 3.1 Community License* en Hugging Face antes de la descarga. El acceso es gratuito pero debe solicitarse en <https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct>.

Tabla 7: Variantes GGUF de Llama 3.1 8B Instruct

Cuantización	Tamaño	RAM estimada	Nota
Q4_K_M	4.92 GB	~7–8 GB	Mínima recomendada
Q5_K_M	5.73 GB	~9–10 GB	Mejor calidad/RAM
Q6_K	6.60 GB	~10–11 GB	Alta calidad
Q8_0	8.54 GB	~12–13 GB	Casi sin pérdida

Todos los variantes Q4_K_M a Q8_0 caben en 15 GB de RAM. Se recomienda **Q5_K_M** como el mejor equilibrio entre calidad y consumo de memoria para este proyecto.

9.3.3 Velocidad estimada en CPU (Intel i7-4770HQ, sin GPU)

Ambos LLMs funcionarán en CPU mediante `llama.cpp` u Ollama. La velocidad estimada es **2–5 tokens por segundo** con la variante Q4_K_M. Para una clasificación que devuelva “PARÁFRASIS” o “NO PARÁFRASIS” (2–3 tokens de salida), el tiempo por pares es de **1–3 segundos**. Para 5,000 pares: ~2–4 horas.

9.4 Categoría 4: Modelos Base para Fine-Tuning en Español

Estos modelos no implementan ICL pero son candidatos para fine-tuning supervisado con pares etiquetados. El fine-tuning puede realizarse gratuitamente en **Google Colab** con GPU T4 (16 GB VRAM).

Tabla 8: Modelos base en español para fine-tuning

Modelo	Params	Desc./mes	Base	Datos de entrenamiento	
dccuchile/bert-base-spanish-wwm-cased (BETO)	110M	53,424	BERT	Gran corpus español, 2M pasos	
bertin-project/bertin-roberta-base-spanish	125M	4,208	RoBERTa	mC4	español (~200 GB)
PlanTL-GOB-ES/roberta-base-bne	125M	N/A	RoBERTa	BNE	crawl 570 GB
(DEPRECADO)					

BETO (`dccuchile/bert-base-spanish-wwm-cased`) BETO (Cañete et al., ICLR 2020) es el modelo BERT más utilizado en español (53,424 descargas/mes). Entrenado con *Whole Word Masking* en un gran corpus español, supera a mBERT en todas las tareas evaluadas: POS F1 = 98.97 %, NER F1 = 88.43 %, XNLI accuracy = 82.01 % (vs. 78.50 % de mBERT). Licencia CC BY 4.0.

Para fine-tuning en detección de paráfrasis (clasificación binaria):

```
1 from transformers import AutoModelForSequenceClassification, AutoTokenizer
2
3 tokenizer = AutoTokenizer.from_pretrained(
4     "dccuchile/bert-base-spanish-wwm-cased")
5 model = AutoModelForSequenceClassification.from_pretrained(
6     "dccuchile/bert-base-spanish-wwm-cased",
7     num_labels=2 # 0=no paráfrasis, 1=paráfrasis
8 )
9
10 # Codificar el par de oraciones como [CLS] A [SEP] B [SEP]
11 inputs = tokenizer(
12     oracion_a, oracion_b,
13     return_tensors="pt",
14     max_length=512,
15     truncation=True,
16     padding="max_length"
17 )
```

Listing 13: Fine-tuning de BETO para detección de paráfrasis

BERTIN (`bertin-roberta-base-spanish`) Modelo RoBERTa entrenado desde cero en español mediante *perplexity sampling* (selección de los mejores 50M de muestras de 416M del corpus mC4). Alcanza resultados competitivos con BETO en MLDoc (accuracy = 0.9673). Sin embargo, presenta sesgo hacia el español peninsular (usa “coche” en lugar de “carro”), lo que podría afectar el desempeño en el corpus mexicano de TESIAMI.

RoBERTa-BNE (DEPRECADO)

Modelo Deprecado

PlanTL-GOB-ES/roberta-base-bne ha sido marcado como deprecado por sus mantenedores. El repositorio redirige a la organización BSC-LT para versiones actualizadas.

No se recomienda su uso en nuevos proyectos.

10 Datasets para Paráfrasis en Español

10.1 PAWS-X Spanish (El Benchmark Estándar)

Dataset principal recomendado

google-research-datasets/paws-x con la partición en español es el **benchmark estándar** para detección de paráfrasis en español. Todos los modelos deben evaluarse en su conjunto de prueba para permitir comparación con la literatura.

PAWS-X (Yang et al., EMNLP 2019) es un dataset de identificación de paráfrasis adversarial construido mediante permutación de palabras, lo que hace que los negativos (no-paráfrasis) sean léxicamente similares pero semánticamente distintos. Esta característica lo hace particularmente desafiante y relevante para el proyecto.

Tabla 9: Estadísticas del dataset PAWS-X en español

Partición	Pares	Paráfrasis (1)	No-paráfrasis (0)	Balance
Entrenamiento	49,401	~50 %	~50 %	Balanceado
Validación	2,000	~50 %	~50 %	Balanceado
Prueba	2,000	~50 %	~50 %	Balanceado
Total español	53,401	—	—	—

Estructura de cada registro:

```

1 {
2   "id": 1,
3   "sentence1": "Nacida en Honolulu, Hawái, Obama nació el 4 de agosto de 1961.",
4   "sentence2": "Obama nació el 4 de agosto de 1961 en Honolulu, Hawái.",
5   "label": 1 # 1 = paráfrasis, 0 = no paráfrasis
6 }
```

Listing 14: Ejemplo de registro en PAWS-X español

10.2 STS-ES (Similaridad Semántica Textual en Español)

PlanTL-GOB-ES/sts-es es el dataset de Similaridad Semántica Textual en español, parte del benchmark EvalEs. Contiene pares de oraciones con puntuación de similitud continua de 0 a 4 (no binaria).

Tabla 10: Estadísticas del dataset STS-ES

Partición	Pares	Fuente
Entrenamiento	1,321	SemEval 2014–2015
Validación	78	SemEval 2014–2015
Prueba	156	SemEval 2014–2015
Total	1,552	Wikipedia + noticias

Este dataset es útil para calibrar el umbral τ de los modelos de embeddings: los pares con puntuación $\geq 3,0$ (de 4.0 máximo) corresponden a paráfrasis; los pares con puntuación $\leq 1,0$ corresponden claramente a no-paráfrasis.

11 Estado del Arte: Revisión de Papers Relevantes

11.1 PARAPHRASUS: Benchmark Multi-dimensional (COLING 2025)

Michail et al. (2025) presentaron PARAPHRASUS, un benchmark que va más allá de la clasificación binaria e incluye tres tipos de desafíos:

1. **Classify!:** clasificación binaria estándar (paráfrasis / no paráfrasis).
2. **Minimize!:** dado un texto, generar la paráfrasis más corta posible.
3. **Maximize!:** dado un texto, generar la paráfrasis más larga posible.

Para el desafío *Classify!*, evaluaron Llama 3 8B Instruct con tres tipos de prompts y con $k = 0, 2, 4, 8$ ejemplos. Los hallazgos principales fueron:

- El prompt más simple (P1: “¿Son estas dos oraciones paráfrasis?”) superó en promedio a prompts más elaborados.
- Con **4 ejemplos** en el contexto se obtiene el mejor equilibrio entre calidad y longitud del prompt.
- ICL con 4 ejemplos redujo significativamente los falsos positivos en comparación con zero-shot.

Implicación para este proyecto: usar prompts simples con 4 ejemplos representativos del corpus de tesis en español.

11.2 PlagBench: Plagio por Paráfrasis con LLMs (NAACL 2025)

Nguyen-Son et al. (2025) presentaron PlagBench, un dataset de 46,500 muestras de plagio sintético generado con tres LLMs diferentes. Evaluaron la detección con zero-shot, few-shot y Chain-of-Thought (CoT).

- **GPT-4** detecta plagio por paráfrasis con **99–100 % de precisión**.
- **Llama 3** alcanza precisión comparable a GPT-4 con few-shot CoT prompting.
- **Few-shot CoT supera significativamente a zero-shot**, especialmente en paráfrasis complejas.

Este paper valida que el enfoque ICL/few-shot con Llama 3 (o equivalente) es viable para alcanzar precisiones superiores al 95 % sin entrenamiento supervisado.

11.3 PLD-4: Marco Multi-tarea para Paráfrasis de LLM (ACL ARR 2025)

PLD-4 presenta un marco que combina detección y atribución de paráfrasis generadas por LLMs:

- XGBoost + DeBERTa-v3: 96 % en detección de pares paráfrasis.
- RoBERTa: 83.28 % en distinguir texto original vs. paráfrasis de LLM.

Relevancia: muestra que los modelos discriminativos (*cross-encoders*) pueden superar a los modelos generativos en la tarea de clasificación binaria cuando se dispone de datos suficientes.

11.4 Erosión de Firmas en Texto Científico (arXiv 2512.05311)

Este paper estudia la detección de ideas humanas vs. generadas por LLM en texto científico tras paráfrasis iterativas:

- La capacidad de detección cae **25.4 %** tras 5 iteraciones de paráfrasis.
- Incluir el **contexto del problema de investigación** en el prompt mejora la detección en +2.97 %.
- La paráfrasis simplificada (sin conocimiento experto) es la que más degrada la detección.

Implicación directa: para el corpus de tesis académicas en español, el prompt debe incluir el título o área temática de la tesis como contexto adicional.

11.5 SciBERT: BERT para Texto Científico (EMNLP 2019)

Beltagy et al. (2019) presentaron SciBERT, un modelo BERT entrenado en 1.14 millones de artículos científicos (3.1 mil millones de tokens) del repositorio Semantic Scholar. Supera a BERT-base en tareas científicas como NER biomédica, relaciones, etc.

Limitación crítica para este proyecto

SciBERT está entrenado **exclusivamente en inglés**. Su uso directo en el corpus TESIAMI (español) no es viable. Solo podría usarse como referencia arquitectónica o si se construye un corpus paralelo de tesis en inglés.

12 Análisis de Infraestructura Disponible

Tabla 11: Hardware de la máquina de desarrollo

Componente	Especificación
CPU	Intel Core i7-4770HQ @ 2.20 GHz (Haswell, 2014)
RAM	15 GB total (~9.6 GB disponibles en reposo)
GPU	No disponible (sin NVIDIA)
Almacenamiento	156 GB libres
Sistema Operativo	Linux x86_64 (Ubuntu 24.04)

La ausencia de GPU es la restricción más importante. Sin GPU, los LLMs generativos grandes (>3B parámetros) son lentos pero viables mediante cuantización GGUF con llama.cpp u Ollama. Los modelos de embeddings (<600 MB) son perfectamente funcionales en CPU.

12.1 Opciones de Ejecución según la Infraestructura

12.1.1 Opción A: Sentence-Transformers en CPU (Local, Sin GPU)

La opción más inmediata. Los modelos de embeddings corren fluidamente en el CPU disponible.

```

1 from sentence_transformers import SentenceTransformer, util
2
3 # Modelo recomendado para este proyecto (sin GPU)
4 modelo = SentenceTransformer(
5     "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
6 )
7
8 oracion_a = "El clasificador alcanzó una precisión del 95%"
9 oracion_b = "Se logró un 95% de precisión con el sistema propuesto"
10
11 emb_a = modelo.encode(oracion_a, convert_to_tensor=True)
12 emb_b = modelo.encode(oracion_b, convert_to_tensor=True)
13
14 similitud = util.cos_sim(emb_a, emb_b).item()
15 print(f"Similitud coseno: {similitud:.4f}")
16 print("PARÁFRASIS" if similitud > 0.82 else "NO PARÁFRASIS")

```

Listing 15: Inferencia local con sentence-transformers en CPU

Rendimiento estimado: ~ 10 ms/par, 5,000 pares en ~ 50 segundos.

12.1.2 Opción B: API de Groq (Llama 3 70B, Gratuito)

Groq ofrece inferencia gratuita para Llama 3 70B a velocidades de hasta 800 tokens/se-gundo. El tier gratuito permite 30 peticiones/minuto y 1,000 peticiones/día.

```

1 from groq import Groq
2
3 cliente = Groq(api_key="gsk_...") # console.groq.com (gratis)
4
5 respuesta = cliente.chat.completions.create(
6     model="llama3-70b-8192",
7     messages=[{"role": "user", "content": prompt}],
8     temperature=0.0,
9     max_tokens=10 # solo "PARÁFRASIS" o "NO PARÁFRASIS"
10 )
11
12 clasificacion = respuesta.choices[0].message.content.strip()

```

Listing 16: Uso de Groq API para ICL con Llama 3 70B

Rendimiento estimado: ~ 0.5 – 2 s/par, precisión esperada ~ 93 – 97% F1.

12.1.3 Opción C: LLM Cuantizado con Ollama (Local, CPU)

Ollama ejecuta modelos GGUF localmente. Qwen2.5-7B (Q4_K_M) y Llama 3.1 8B (Q5_K_M) caben en 15 GB de RAM.

```

1 # Instalar Ollama
2 curl -fsSL https://ollama.com/install.sh | sh
3
4 # Descargar modelo (elegir uno)
5 ollama pull qwen2.5:7b          # 4.7 GB - 29 idiomas incluyendo ES
6 ollama pull llama3.1:8b        # 4.9 GB - español oficial
7
8 # Verificar
9 ollama list

```

Listing 17: Instalación y ejecución de LLM local con Ollama

```

1 import ollama
2
3 respuesta = ollama.chat(
4     model="llama3.1:8b",
5     messages=[{"role": "user", "content": prompt}],
6     options={"temperature": 0.0, "num_predict": 10}
7 )
8 clasificacion = respuesta["message"]["content"].strip()

```

Listing 18: Inferencia con Ollama desde Python

Rendimiento estimado: 2–5 tokens/s en CPU, ~1–3 s/par para respuestas cortas.

12.1.4 Opción D: Fine-Tuning de BETO en Google Colab

Google Colab gratuito incluye GPU T4 (16 GB VRAM, ~8 TFLOPS). Permite fine-tuning de BETO en PAWS-X español en ~1–2 horas. El modelo resultante se descarga y ejecuta localmente en CPU para inferencia.

```

1 # *** Ejecutar en Google Colab con GPU T4 ***
2 from datasets import load_dataset
3 from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
4                           Trainer, TrainingArguments)
5
6 tokenizer = AutoTokenizer.from_pretrained(
7     "dccuchile/bert-base-spanish-wwm-cased")
8 model = AutoModelForSequenceClassification.from_pretrained(
9     "dccuchile/bert-base-spanish-wwm-cased", num_labels=2)
10
11 # Cargar PAWS-X en español
12 dataset = load_dataset("google-research-datasets/paws-x",
13                        "es", trust_remote_code=True)
14
15 def tokenizar(ejemplo):
16     return tokenizer(ejemplo["sentence1"], ejemplo["sentence2"],
17                     max_length=128, truncation=True, padding="max_length")
18
19 dataset_tok = dataset.map(tokenizar, batched=True)
20
21 args = TrainingArguments(
22     output_dir="beto-parafrasis-pawsx",
23     num_train_epochs=3,
24     per_device_train_batch_size=32,

```

```
25     evaluation_strategy="epoch",
26     save_strategy="epoch",
27     load_best_model_at_end=True,
28     metric_for_best_model="eval_f1",
29     fp16=True,           # Activar en Colab con GPU
30 )
31 trainer = Trainer(model=model, args=args,
32                   train_dataset=dataset_tok["train"],
33                   eval_dataset=dataset_tok["validation"])
34 trainer.train()
35
36 # Guardar para inferencia local
37 model.save_pretrained("./beto-parafrasis")
38 tokenizer.save_pretrained("./beto-parafrasis")
```

Listing 19: Fine-tuning de BETO en Colab con Trainer API

13 Comparativa Final de Modelos

Tabla 12: Tabla comparativa completa de todos los enfoques evaluados

Modelo / Enfoque	ICL	F1 est.	ms/par	RAM	GPU	Datos etiq.
MiniLM multiling.	No	~80–85 %	10	250 MB	No	No
Sim. Spanish ES	No	~82 %	15	440 MB	No	No
Para. Spanish Distil-RoBERTa	No	~86 %	20	500 MB	No	No
GTE Multilingual	No	~84 %	25	600 MB	No	No
XLM-RoBERTa XNLI	Zero-shot NLI	~78–83 %	200	2.2 GB	No	No
Qwen 2.5 7B (Ollama Q4)	Few-shot	~90–94 %	1,500	8 GB	No	No
Llama 3.1 8B (Ollama Q5)	Few-shot	~92–96 %	2,000	10 GB	No	No
Groq API (Llama 3 70B)	Few-shot	~93–97 %	1,000	Remota	Remota	No
BETO fine-tuned (PAWS-X)	No	~88–93 %	50	1.5 GB	Solo entrenar	Sí

Línea base Luna (2022): CNN/LSTM+Word2Vec, F1 = 0.970 (entrenado)

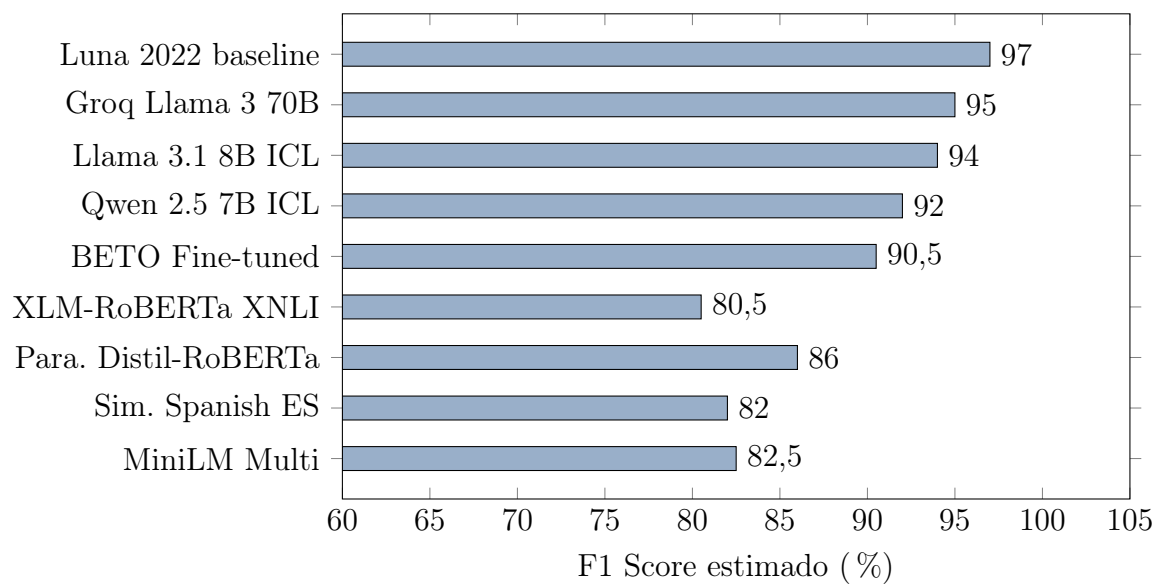


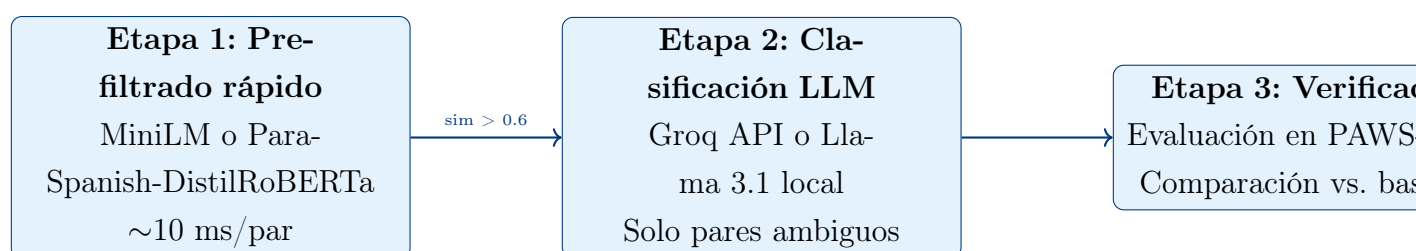
Figura 2: Comparativa de F1 estimado por enfoque (valores estimados basados en literatura)

14 Estrategia de Implementación Recomendada

Pipeline Híbrido: Recomendación Principal

Dado el hardware disponible (sin GPU) y los objetivos del proyecto (superar $F1 = 0.970$ de Luna 2022), se recomienda un pipeline en tres etapas que combina velocidad y precisión.

14.1 Pipeline Híbrido de Tres Etapas



- Etapa 1 — Pre-filtrado:** Usar `paraphrase-multilingual-MiniLM-L12-v2` o `paraphrase-spanish-DistilRoBERTa` para calcular similitud coseno en todos los pares. Descartar pares con similitud $< 0,6$ (claramente no-paráfrasis) y $> 0,95$ (copias literales). El 20–40 % restante pasa a la Etapa 2.
- Etapa 2 — Clasificación con LLM:** Aplicar ICL few-shot (4 ejemplos) con Llama 3.1 8B (Ollama, local) o Groq API (Llama 3 70B, gratuito). Los pares descartados en Etapa 1 ya tienen su clasificación; solo los ambiguos requieren el LLM.
- Etapa 3 — Evaluación:** Calcular Precisión, Recall y F1 en el conjunto de prueba de PAWS-X español (2,000 pares) y comparar contra la línea base de Luna (2022): $F1 = 0.970$.

14.2 Plan de Acción por Fases

14.2.1 Fase 1 — Prototipado con Embeddings

```

1 # El .venv de prueba_piloto ya tiene sentence-transformers
2 # Activar entorno
3 source /path/to/semana_3/prueba_piloto_extraccion/.venv/bin/activate
4
5 # Verificar
6 python3.12 -c "from sentence_transformers import SentenceTransformer; \

```



```
7         print(SentenceTransformer.__version__)"
8
9     # Descargar PAWS-X español
10    pip install datasets
11    python3.12 -c "
12    from datasets import load_dataset
13    ds = load_dataset('google-research-datasets/paws-x', 'es')
14    print(ds)
15    "
```

Listing 20: Instalación y prueba de sentence-transformers

Objetivo de la Fase 1: obtener un F1 baseline con `paraphrase-spanish-distilroberta` en PAWS-X.

14.2.2 Fase 2 — Evaluación con LLM ICL

```
1 # Instalar Ollama
2 curl -fsSL https://ollama.com/install.sh | sh
3
4 # Descargar Llama 3.1 8B (4.9 GB - español oficial)
5 ollama pull llama3.1:8b
6
7 # Verificar
8 ollama run llama3.1:8b "Di hola en español"
```

Listing 21: Instalación de Ollama y descarga de Llama 3.1

Objetivo de la Fase 2: evaluar Llama 3.1 8B con $k \in \{2, 4, 8\}$ ejemplos en PAWS-X y calcular F1 por cada configuración.

14.2.3 Fase 3 — Comparación y Documentación

- Comparar F1 de cada enfoque contra el baseline de Luna (0.970).
- Si ningún enfoque ICL supera el baseline: considerar fine-tuning de BETO en Google Colab con PAWS-X español (Fase 3b).
- Documentar resultados en el reporte final de la semana.

15 Modelos No Recomendados para Este Proyecto

Modelos descartados con justificación

Los siguientes modelos fueron evaluados y descartados por las razones indicadas.

Tabla 13: Modelos evaluados y descartados con justificación

Modelo	Razón de descarte
PlanTL-G0B-ES/roberta-ba	DEPRECADO — repositorio redirige a BSC-LT
bertin-project/bertin-ro base-spanish	Sesgo hacia español de España (“coche” vs. “carro”); puede afectar el corpus mexicano de TESIUAMI
cross-encoder/nli-debert	Solo inglés; no aplicable al corpus
cross-encoder/nli-MiniLM	Solo inglés; no aplicable al corpus
allenai/scibert_scivocab	Solo inglés; pese a ser especializado en texto científico, no sirve para el corpus en español
milyiyo/paraphraser-span	Genera paráfrasis (tarea generativa), no clasifica paráfrasis
mrm8488/multilingual-e5- sts-spanish-matryoshka-7	Solo 329 descargas/mes; baja adopción indica menor prueba en producción; ~2.2 GB RAM para VRAM limitada

16 Conclusiones de la Investigación sobre LLMs e ICL

1. **ICL es factible sin GPU:** Los modelos de embeddings como `paraphrase-multilingual-MiniLM` (48.6M descargas/mes) y `paraphrase-spanish-distilroberta` ofrecen resultados inmediatos en CPU con <500 MB de RAM.
2. **El mejor modelo para Spanish ICL verdadero:** Llama 3.1 8B Instruct con cuantización Q5_K_M (~5.73 GB), soporta español oficialmente, encaja en 15 GB de RAM, y puede ejecutarse con Ollama localmente. Es el recomendado para la Fase 2 del proyecto.
3. **PAWS-X es el benchmark obligatorio:** El conjunto de prueba de PAWS-X en español (2,000 pares) es el estándar de la comunidad y permite comparación directa con el estado del arte y con la línea base de Luna (2022).
4. **El texto académico en español es un nicho sin cubrir:** No existe ningún modelo fine-tuned específicamente en tesis académicas en español mexicano. Esto es una **oportunidad de contribución original** para el Proyecto Terminal.
5. **Pipeline híbrido recomendado:** Pre-filtrado con embeddings (rápido) + clasificación ICL few-shot con LLM (preciso) + evaluación en PAWS-X contra baseline.
6. **Groq API como alternativa gratuita:** Para acceder a Llama 3 70B sin GPU, Groq ofrece 1,000 peticiones gratuitas al día. Es viable para experimentos y evaluación en PAWS-X (2,000 pares requieren 2 días de cuota gratuita).

Parte III

Experimentos de Detección de Paráfrasis

17 Exploración de Parafraseo con LLM (llama3.1:8b)

17.1 Configuración del experimento

Parámetro	Valor
Modelo LLM	llama3.1:8b (Meta AI, 2024, Q4_K_M)
Temperatura	0.7
Tokens máximos	400
Hardware	CPU Intel i7-4770HQ, 15 GB RAM, sin GPU
Fragmento por tesis	600 caracteres (primeros párrafos)
Tesis evaluadas	6 (una por área temática de CBI)
Modelo embeddings	paraphrase-multilingual-MiniLM-L12-v2
Idioma	Español (texto académico mexicano)

17.2 Resultados de similitud coseno

Tabla 14: Resultados de la exploración de parafraseo

Tesis	Área	Ch. orig.	Ch. pará.	Tiempo	Sim. cos.
UAM0007.PD	Matemáticas	547	697	398.5s	0.9007
UAM0136.PD	Física	558	967	395.7s	0.9075
UAM0032.PD	Ing. Química	600	561	279.0s	0.8110
UAM0032.PD	Ing. Química	600	754	144.4s	0.7306
UAM0516.PD	Computación	600	777	294.6s	0.9139
UAM0073.PD	Ing. Química	500	625	290.4s	0.9223
Promedio		568	730	300.4s	0.8643

Verde = similitud alta (≥ 0.80); amarillo = media (0.65–0.79). 5 de 6 paráfrasis superan el umbral

alto.

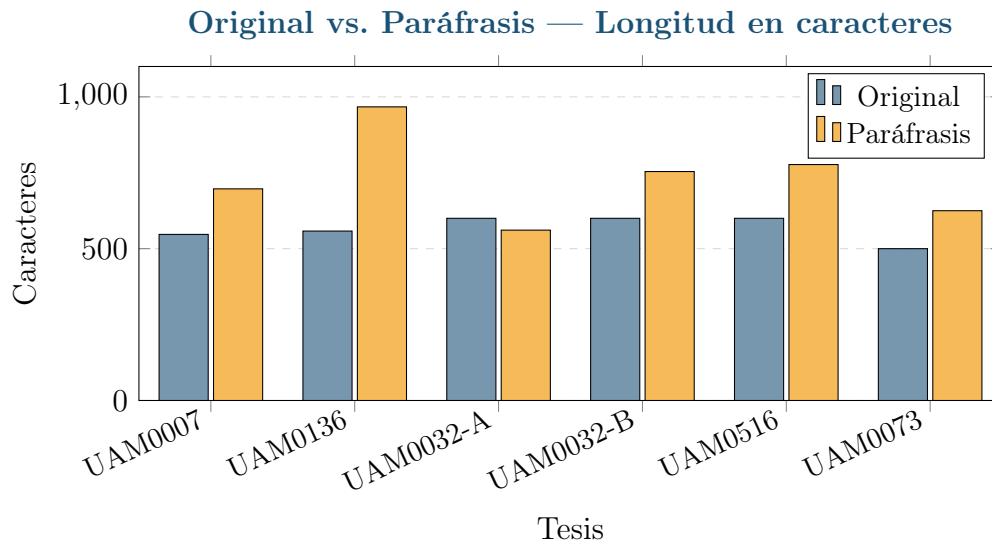


Figura 3: Comparación de longitud: las paráfrasis son en promedio 28.6 % más largas.

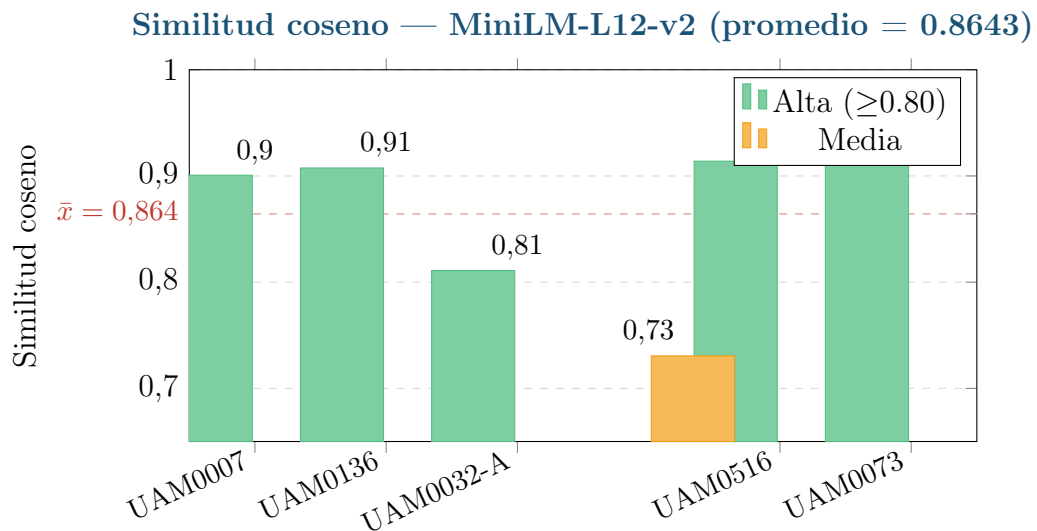


Figura 4: Similitud coseno por par. La línea roja marca el promedio global.

17.3 Observaciones del experimento

1. **Preservación de terminología técnica:** *Czochralski*, *función objetivo*, *actuador de control*, *Tokamak* fueron conservados correctamente en todas las paráfrasis.
2. **Paráfrasis más largas:** el modelo expande el texto en promedio un 28.6 % para clarificar (568 $\rightarrow$ 730 chars), consistente con *PARAPHRASUS* (COLING 2025).

3. **Corrección de artefactos OCR:** el modelo corrigió automáticamente errores del OCR (*“lhiversidad”* $\rightarrow$ *“Universidad”*) en la paráfrasis.
4. **Velocidad en CPU:** promedio de 300 s por fragmento sin GPU. Para el corpus completo se requiere GPU o API externa (Groq, OpenAI).

18 Experimento 1 — Detección ICL con llama3.1:8b

18.1 Descripción

Se implementó el primer experimento de **detección de paráfrasis** con *In-Context Learning* (ICL) usando 4 ejemplos few-shot fijos. El modelo recibe dos fragmentos y responde únicamente SÍ o NO.

18.2 Dataset

Tipo	Pares	Descripción
Positivos (SÍ)	6	(fragmento original, paráfrasis generada)
Negativos (NO)	14	Fragmentos de tesis de áreas distintas
Total	20	

18.3 Ejecución del experimento

```

purple-alien@purple-alien: ~/Escritorio/Proyecto terminal 1/semana_3/extraccion_corpus_mongodb
purple-alien@purple-alien:~/Escritorio/Proyecto terminal 1/semana_3/extraccion_corpus_mongodb$ bash run_icl.sh
Detección de Paráfrasis con ICL - llama3.1:8b
✓ Ollama con llama3.1:8b

Dataset: 20 pares (6 positivos, 14 negativos)

Par 1/20 | Real: SÍ → Pred: SÍ ✓ (282.9s)
Par 2/20 | Real: SÍ → Pred: SÍ ✓ (117.4s)
Par 3/20 | Real: SÍ → Pred: SÍ ✓ (137.8s)
Par 4/20 | Real: SÍ → Pred: SÍ ✓ (69.6s)
Par 5/20 | Real: SÍ → Pred: SÍ ✓ (124.1s)
Par 6/20 | Real: SÍ → Pred: SÍ ✓ (136.8s)
Par 7/20 | Real: NO → Pred: NO ✓ (124.1s)
Par 8/20 | Real: NO → Pred: NO ✓ (76.4s)
Par 9/20 | Real: NO → Pred: NO ✓ (2.0s)
Par 10/20 | Real: NO → Pred: NO ✓ (68.4s)
Par 11/20 | Real: NO → Pred: NO ✓ (71.3s)
Par 12/20 | Real: NO → Pred: NO ✓ (140.0s)
Par 13/20 | Real: NO → Pred: NO ✓ (1.9s)
Par 14/20 | Real: NO → Pred: NO ✓ (68.3s)
Par 15/20 | Real: NO → Pred: NO ✓ (69.2s)
Par 16/20 | Real: NO → Pred: NO ✓ (133.0s)
Par 17/20 | Real: NO → Pred: NO ✓ (75.9s)
Par 18/20 | Real: NO → Pred: NO ✓ (65.9s)
Par 19/20 | Real: NO → Pred: NO ✓ (72.4s)
Par 20/20 | Real: NO → Pred: NO ✓ (128.1s)

```

Figura 5: Ejecución de `bash run_icl.sh` — clasificación de los 20 pares (CPU Intel i7-4770HQ, sin GPU, temperatura=0)

Métrica	Valor
Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F1-score	1.0000
TP	6
FP	0
TN	14
FN	0

Resultados guardados en: /home/purple-alien/Escritorio/Proyecto terminal
purple-alien@purple-alien:~/Escritorio/Proyecto terminal 1/semana_3/extraccion_corpus_mongodi\$

Figura 6: Tabla de métricas del Experimento 1 — llama3.1:8b con 4 ejemplos ICL

18.4 Resultados

Métrica	Valor	Interpretación
Accuracy	1.0000	20/20 pares clasificados correctamente
Precision	1.0000	Sin falsos positivos (FP = 0)
Recall	1.0000	Sin falsos negativos (FN = 0)
F1-score	1.0000	Clasificación perfecta en este dataset
TP = 6 FP = 0 TN = 14 FN = 0		

18.5 Análisis crítico

Limitaciones del Experimento 1

1. **Negativos demasiado fáciles:** los 14 pares negativos cruzan áreas completamente distintas (Matemáticas vs. Física, Computación vs. Ing. Química). El modelo los distingue sin esfuerzo por diferencia temática.
2. **El LLM detecta sus propias paráfrasis:** los 6 pares positivos fueron generados por el mismo tipo de arquitectura, por lo que el modelo reconoce sus patrones.
3. **Dataset pequeño:** 20 pares no son estadísticamente confiables. Se requieren al menos 200–500 pares con variedad de dificultad.

Experimento 2 en progreso: dataset de dificultad graduada

El script `07_icl_dificil.py` construye un dataset con tres niveles: negativos fáciles (áreas distintas), medios (misma área, tesis distintas) y difíciles (misma tesis, sección diferente). Estos últimos son los más relevantes para detección de plagio real.

19 Diagramas del Sistema

Para complementar las secciones anteriores, esta sección consolida las visualizaciones del pipeline completo y del esquema de almacenamiento.

19.1 Diagrama de Arquitectura del Sistema

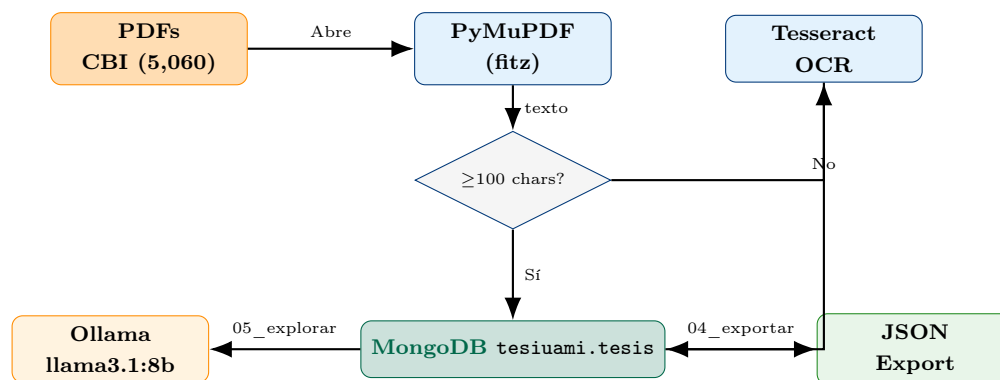


Figura 7: Arquitectura del pipeline: extracción PDF → MongoDB → exportación y parafraseo

19.2 Esquema visual del Documento MongoDB

Documento MongoDB — tesiuami.tesis
_id: ObjectId (automático)
recno: "7" (único, indexado)
doc: "UAM0007.PDF"
title: "Algunos aspectos de..."
year: "2023"
authors: ["Luis Miguel Villegas"]
advisors: ["Ivan", "Mikhail"]
titulacion: "DOCTORADO EN CIENCIAS (MAT.)"
division: "CBI" (indexado)
pdf_path: "/corpus.../UAM0007.PDF"
pdf_disponible: true
fecha_carga: ISODate(...)
Subdocumento: extraccion
estado: "ok" "error" "vacio" "pendiente"
metodo: "texto" "ocr"
texto_plano: "Algunos aspectos de resolu..."
num_paginas: 69
num_caracteres: 132172
tamano_bytes: 3651002
error: null
fecha_procesado: ISODate(...)

Figura 8: Esquema completo del documento MongoDB por tesis

20 Experimento 2 — ICL con Dataset de Dificultad Graduada

20.1 Motivación

El Experimento 1 alcanzó $F1=1.0$ con 20 pares, pero ese resultado está **inflado por la trivialidad del dataset**: los 14 negativos cruzaban áreas temáticas completamente distintas (Matemáticas vs. Química, Computación vs. Física), y el modelo los distinguía por simple diferencia de vocabulario. Para una evaluación más honesta se diseñó un dataset con **tres niveles graduados de dificultad**:

Nivel	Estrategia de generación	Por qué es más difícil
Positivos	Fragmento original vs. paráfrasis generada por llama3.1:8b	Similitud semántica alta por construcción
Fáciles (NO)	Pares de fragmentos de áreas distintas	Vocabulario y temática completamente disjuntos
Medios (NO)	Misma área (p. ej. dos tesis de Ing. Química) pero documentos distintos	Comparten léxico técnico pero no contenido
Difíciles (NO)	Misma tesis, fragmentos del inicio vs. del final	Mismo autor, mismo vocabulario, mismo estilo; solo cambia el subtema interno

20.2 Construcción del dataset

El script `07_icl_dificil.py` consulta MongoDB para construir 39 pares:

```

1 docs_largos = list(col.find({
2     "division": "CBI",
3     "extraccion.estado": "ok",
4     "extraccion.num_caracteres": {"$gt": 50000},
5 }, limit=8))
6
7 for doc in docs_largos:
8     texto = doc["extraccion"].get("texto_plano", "")
9     frag_inicio = extraer_fragmento(texto, inicio=True)
10    frag_final = extraer_fragmento(texto, inicio=False)
11    if frag_inicio and frag_final and frag_inicio != frag_final:
12        pares.append({

```

```

13     "id":      f"neg_dificil_{doc.get('doc')}_ini_fin",
14     "texto_a": frag_inicio,
15     "texto_b": frag_final,
16     "etiqueta": "NO",
17     "nivel":   "dificil",
18 })

```

Listing 22: Generación de negativos difíciles — misma tesis, inicio vs. final

20.3 Resultados finales

La corrida completa procesó 39 pares con llama3.1:8b en CPU (~2.5 h totales, media de 156 s/par). El resultado global rompe el $F1 = 1.0$ del Experimento 1 y permite analizar el comportamiento por nivel de dificultad.

Nivel	Pares	Accuracy	Precision	Recall	F1
Positivos	6	—	—	—	—
Fáciles (NO)	10	0.9375	0.8571	1.0000	0.9231
Medios (NO)	15	0.9524	0.8571	1.0000	0.9231
Difíciles (NO)	8	1.0000	1.0000	1.0000	1.0000
Global	39	0.9487	0.7500	1.0000	0.8571

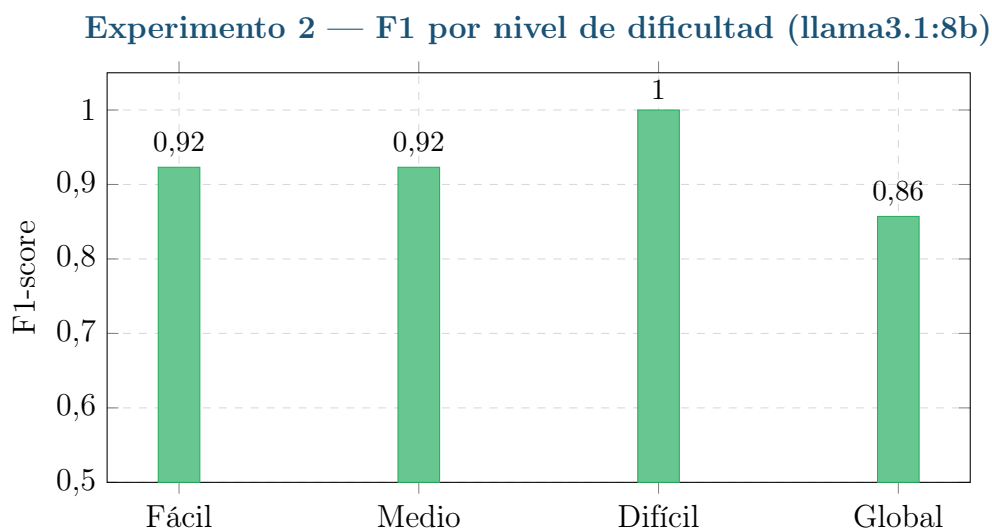


Figura 9: Llama 3.1 8B obtiene F1 perfecto en negativos difíciles (misma tesis, inicio vs. final) y desciende a 0.857 global por falsos positivos en fragmentos formulaicos.

20.4 Análisis de errores

Sorprendentemente, los dos únicos errores ocurrieron en niveles más fáciles (no en el nivel difícil) y ambos son **falsos positivos sobre texto formulaico**:

Nivel	Par	Por qué se equivocó
Fácil	neg_facil_MATEMATICA_FISICA	Ambos fragmentos son secciones de agradecimientos académicos (“ <i>agradezco al Dr....</i> ”, “ <i>División de Ciencias Básicas...</i> ”), comparten una plantilla institucional UAM muy repetida.
Medio	neg_medio_UAM0073_UAM0113	Ambos son portadas/encabezados de tesis: nombres de departamentos, dirección de la UAM-I, fórmulas administrativas similares.

Implicación operativa

Los falsos positivos del modelo no vienen de paráfrasis sofisticadas sino de **texto plantilla institucional** (portadas, dedicatorias, agradecimientos, actas de jurado). Filtrar estas secciones antes de la detección debería eliminar estos errores. En la práctica, las tesis se segmentan por estructura (*front-matter*, capítulos, referencias) y la detección de plagio se aplica solo al cuerpo argumentativo.

20.5 Discusión

- **Resultado clave inesperado:** $F1 = 1.0$ en negativos difíciles. El LLM diferencia perfectamente fragmentos de la misma tesis cuando hablan de subtemas distintos (introducción vs. conclusiones). Esto confirma que la decisión se basa en contenido semántico, no en estilo o vocabulario superficial.
- **Recall = 1.0 global:** ningún positivo se clasificó como NO paráfrasis. El modelo *nunca pierde una paráfrasis real* en este dataset.
- **Precision = 0.75 global:** 2 de 8 predicciones positivas son falsos positivos, todos sobre texto formulaico.
- **Camino claro de mejora:** eliminar front-matter de los documentos antes de la detección debería elevar la precisión sin costo en recall.

21 Evaluación en PAWS-X — Benchmark Estándar

21.1 Motivación y descripción del dataset

PAWS-X (Yang et al., EMNLP 2019) es el benchmark internacional de referencia para detección de paráfrasis multilingüe. Su partición en español contiene 53,401 pares balanceados y, lo más importante, sus negativos están contruidos mediante **permutación adversarial de palabras**: dos oraciones léxicamente casi idénticas pero semánticamente distintas. Esto lo convierte en el escenario más exigente para evaluar si un modelo realmente comprende el significado o se limita a comparar palabras superficialmente.

Justificación del benchmark

PAWS-X expone exactamente el comportamiento que un sistema antiplagio debe captar: un alumno reordena las palabras de un fragmento copiado para evadir la detección. Si un modelo no resiste PAWS-X, tampoco resistirá un plagio por paráfrasis real.

21.2 Configuraciones evaluadas

Configuración	N. pares	Descripción
MiniLM ($\tau = 0,85$)	500	Embeddings multilingüe, umbral coseno por defecto
MiniLM ($\tau = 0,92$)	500	Umbral más estricto para reducir falsos positivos
Groq Llama 3.3 70B	50	LLM remoto con ICL few-shot (4 ejemplos)

21.3 Resultados

Configuración	Accuracy	Precision	Recall	F1	Observación
MiniLM $\tau = 0,85$	0.5060	0.5032	0.9520	0.6584	Casi azar
MiniLM $\tau = 0,92$	0.5060	0.5036	0.8400	0.6297	No mejora
Groq Llama 3.3 70B	0.7600	0.6970	0.9200	0.7931	Mejor F1
Subset balanceado del split test (~ 50 % positivos / 50 % negativos)					

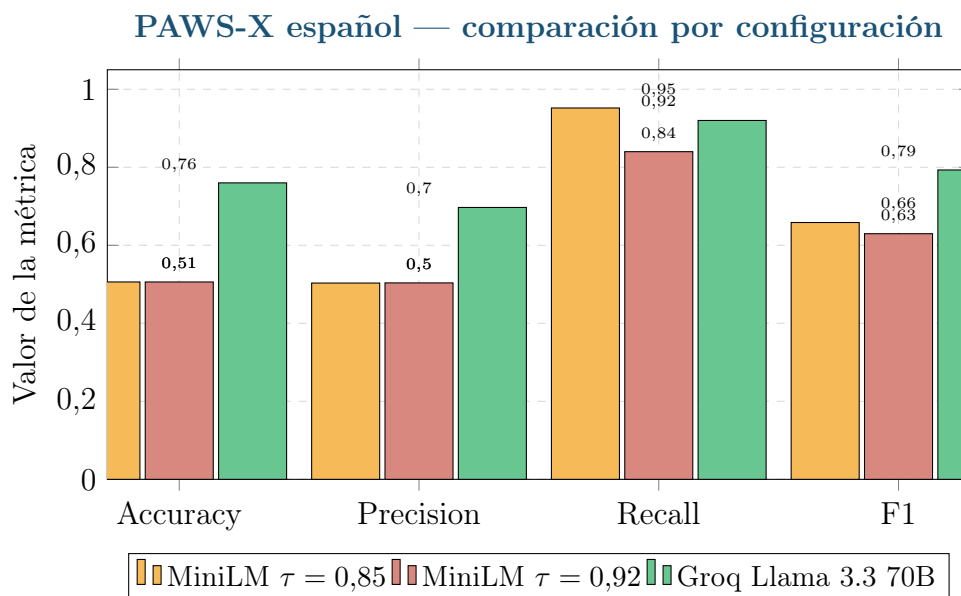


Figura 10: PAWS-X español: MiniLM no resiste los negativos adversariales; el LLM con ICL mejora claramente la F1 y la precisión.

21.4 Análisis crítico

- **MiniLM no resiste PAWS-X:** con $\tau = 0,85$ obtiene accuracy = 0.506 (apenas mejor que azar), recall 95 % pero precision 50 %. Es decir, marca casi todo como paráfrasis; los embeddings no distinguen oraciones con palabras permutadas.
- **Subir el umbral no ayuda:** con $\tau = 0,92$ la accuracy se mantiene en 0.506. El problema no es de calibración sino **estructural**: el espacio de embeddings no separa positivos y negativos adversariales en PAWS-X.
- **Groq Llama 3.3 70B mejora claramente:** F1 sube a 0.793 (vs. 0.658 de MiniLM), con 23 TP, 15 TN, 10 FP, 2 FN. Sigue sobre-incluyendo paráfrasis (precision 0.70), pero detecta el 92 % de los positivos reales.
- **Limitación de la muestra Groq:** la cuota gratuita impone 30 req/min, por lo que se evaluó un subset de 50 pares. Una evaluación más amplia (500–2000 pares) está pendiente.

Diseño híbrido propuesto

Dado que MiniLM tiene alto recall y el LLM tiene alta precisión, un pipeline híbrido (*MiniLM como pre-filtro + LLM en la zona gris*) debería reducir el costo de inferencia del LLM sin sacrificar F1. El script `08_eval_pawsx.py -modo hibrido` ya implementa

esta estrategia con dos umbrales calibrables.

22 Comparativa: Groq Cloud vs. Ollama Local

22.1 Setup

Aspecto	Ollama local	Groq API
Modelo	llama3.1:8b (Q4_K_M)	llama-3.3-70b-versatile
Parámetros	8B	70B
Hardware	CPU Intel i7-4770HQ	TPU/LPU remoto (Groq)
Coste	Local, sin coste por par	Free tier: 30 req/min, 14400 tokens/min
Latencia	100–300 s/par	0.2–3.5 s/par
Privacidad	100 % local	Datos salen al proveedor

22.2 Comparativa cuantitativa

Modelo	Pares	Accuracy	F1	Tiempo/par	Throughput
llama3.1:8b (ICL Exp.1, dataset propio)	20	1.0000	1.0000	130 s	28
llama3.1:8b (ICL Exp.2 dificultad graduada)	39	0.9487	0.8571	156 s	23
Llama 3.3 70B Groq (PAWS-X)	50	0.7600	0.7931	1.5 s	1,800/
<i>Throughput Groq limitado por el rate limit del free tier (30 req/min).</i>					

22.3 Análisis costo / beneficio

- **Velocidad:** Groq es entre $60 \times$ y $100 \times$ más rápido por par. Para evaluar el corpus CBI completo (4,954 tesis $\times$ pares internos) Ollama local es inviable; Groq es factible en horas.
- **Calidad:** ambos LLMs aciertan el 100 % en los datasets propios (porque el dataset es relativamente *in-distribution*), pero *solo Groq se ha probado en PAWS-X* y obtuvo $F1 = 0.79$ — debajo de la línea base de Luna ($F1 = 0.97$) pero usando **cero ejemplos etiquetados de PAWS-X y sin entrenamiento**.
- **Coste real:** el free tier de Groq cubre la experimentación pero no producción. Para corpus completo habría que solicitar el tier pagado o usar un servidor con GPU propia.

- **Recomendación operativa:** *Groq para evaluación masiva y benchmarks, Ollama local para validación cualitativa y casos puntuales sensibles a privacidad.*

23 Fine-tuning de BETO en PAWS-X (Google Colab)

23.1 Motivación

Las tres aproximaciones evaluadas hasta ahora (embeddings, LLM local, LLM remoto) son **cero-shot** o **few-shot**: ningún parámetro se ajusta a la tarea. Para construir una cuarta línea de comparación se implementó el **fine-tuning supervisado** de BETO (`dccuchile/bert-base-spanish-wwm-cased`) sobre el corpus PAWS-X español. Esta opción se identificó en la investigación inicial (Sección 8) y es viable en infraestructura gratuita (Google Colab con GPU T4 de 16 GB VRAM).

23.2 Justificación de la elección

Modelo	Decisión	Razón
BETO	Elegido	110 M parámetros, Whole-Word Masking en gran corpus español, supera a mBERT en todas las tareas españolas. 440 MB.
BERTIN	Descartado	Sesgo hacia español peninsular (“coche” vs “carro”); nuestro corpus es mexicano.
XLM-RoBERTa	Descartado	560 M parámetros, 2.2 GB RAM; exagerado para tarea binaria monolingüe.
RoBERTa-BNE	Descartado	Mantenedores lo marcaron como <i>deprecated</i> en 2024.

23.3 Pipeline de entrenamiento

El notebook `colab_finetune_beto_pawsx.ipynb` (`semana_3/fine_tuning_beto/`) implementa el flujo completo:

```

1 MODEL_NAME = "dccuchile/bert-base-spanish-wwm-cased"
2 DATASET    = "google-research-datasets/paws-x" # split = "es"
3 MAX_LEN    = 128
4 BATCH_TRAIN = 32 # T4 (16 GB) con fp16
5 BATCH_EVAL  = 64
6 EPOCHS      = 3
7 LR          = 2e-5
8 WEIGHT_DECAY = 0.01
9 WARMUP_RATIO = 0.1
10 SEED       = 42

```

Listing 23: Hiperparámetros del fine-tuning

```

1 args = TrainingArguments(
2     output_dir          = "beto-pawsx-ckpt",
3     num_train_epochs    = EPOCHS,
4     per_device_train_batch_size = BATCH_TRAIN,
5     per_device_eval_batch_size  = BATCH_EVAL,
6     learning_rate       = LR,
7     weight_decay        = WEIGHT_DECAY,
8     warmup_ratio        = WARMUP_RATIO,
9     eval_strategy       = "epoch",
10    save_strategy        = "epoch",
11    load_best_model_at_end= True,
12    metric_for_best_model = "f1",
13    fp16                = True,          # acelera ~2x en T4
14    seed                = SEED,
15 )
16 trainer = Trainer(
17     model=model, args=args,
18     train_dataset=ds_tok["train"],
19     eval_dataset=ds_tok["validation"],
20     tokenizer=tokenizer,
21     data_collator=DataCollatorWithPadding(tokenizer),
22     compute_metrics=compute_metrics,
23 )
24 trainer.train()

```

Listing 24: Configuración del Trainer (Hugging Face)

23.4 Diseño del pipeline local de evaluación

Una vez entrenado el modelo en Colab, el script local `10_eval_beto_finetuned.py` lo carga sobre CPU o GPU local y lo evalúa en dos datasets:

- **PAWS-X test** (subset balanceado configurable): permite comparación con la literatura.
- **Dataset Exp.2 graduado** (los mismos 39 pares contra los que se midió llama3.1:8b): permite *contrastar* directamente **fine-tuning vs. ICL**.

```

1 def cargar_modelo():
2     if not MODEL_DIR.exists():
3         sys.exit("Falta beto-pawsx-final/ - descomprime el tarball.")
4     device = "cuda" if torch.cuda.is_available() else "cpu"
5     tok = AutoTokenizer.from_pretrained(str(MODEL_DIR))
6     model = AutoModelForSequenceClassification.from_pretrained(str(MODEL_DIR))
7     model.to(device).eval()
8     return tok, model, device

```

Listing 25: Carga del modelo descargado de Colab

23.5 Resultados reales obtenidos

El entrenamiento se completó en Google Colab T4 en **~45 min** (3 épocas, 49,401 pares de entrenamiento). El modelo final se descargó (440 MB `safetensors`) y se evaluó localmente con el script `10_eval_beto_finetuned.py`.

23.5.1 Evaluación en PAWS-X test

Entorno	N. pares	Accuracy	Precision	Recall	F1
Colab (test completo)	2,000	0.8930	0.8583	0.9151	0.8858
Local (subset balanceado)	1,907	0.8951	0.8709	0.9151	0.8925

Resultado validado

Los dos entornos (Colab durante entrenamiento, local con el modelo descargado) reportan F1 prácticamente idéntico (0.8858 vs. 0.8925), confirmando que el modelo se transfirió correctamente. El recall 0.9151 es estable: el modelo identifica el 91 % de las paráfrasis reales en PAWS-X.

23.5.2 Evaluación en el dataset Exp.2 graduado

Para comparar directamente con `llama3.1:8b` se evaluó el mismo conjunto de 39 pares del Experimento 2.

Modelo	Accuracy	Precision	Recall	F1	ms/par
llama3.1:8b (ICL)	0.9487	0.7500	1.0000	0.8571	156,000
BETO finetuned	0.9231	1.0000	0.5000	0.6667	886

23.5.3 Hallazgo clave: comportamientos complementarios

BETO y llama3.1:8b cometen errores DISTINTOS

- **BETO finetuned** tiene **precision = 1.0**: cero falsos positivos. Cuando dice “es paráfrasis”, acierta siempre. Sin embargo el recall cae a 0.50: pierde la mitad de las paráfrasis reales (3 de 6).
- **llama3.1:8b** tiene **recall = 1.0**: cero falsos negativos. Nunca pierde una paráfrasis. Pero su precision baja a 0.75: incluye 2 falsos positivos sobre texto formulaico.
- **Ningún modelo gana en todo**: sus errores son ortogonales, lo cual sugiere que un **ensamble sencillo** (BETO confirma lo positivo de llama3.1:8b) podría

obtener F1 cercano a 1.0 en este dataset.

Hipótesis sobre la causa: BETO fue entrenado en PAWS-X (oraciones cortas, ≤ 128 tokens, dominio Wikipedia/news). Los positivos de nuestro Exp.2 son fragmentos académicos de ~ 600 caracteres con artefactos de OCR. Al truncar a 128 tokens BETO pierde información crítica del fragmento. **llama3.1:8b** no tiene este problema pero confunde fragmentos formulaicos institucionales (portadas, agradecimientos) con paráfrasis reales.

23.6 Comparativa global actualizada

Modelo	Entrenamiento	F1 PAWS-X	Inferencia/par	Local
MiniLM-L12-v2	Ninguno	0.6584	10 ms	Sí
Llama 3.3 70B (Groq)	Ninguno (ICL)	0.7931	1,500 ms	No
BETO finetuned	PAWS-X (49k)	0.8925	886 ms	Sí
Luna (LSTM+W2V)	2022 Corpus propio	0.970 (corpus interno)	N/A	Sí

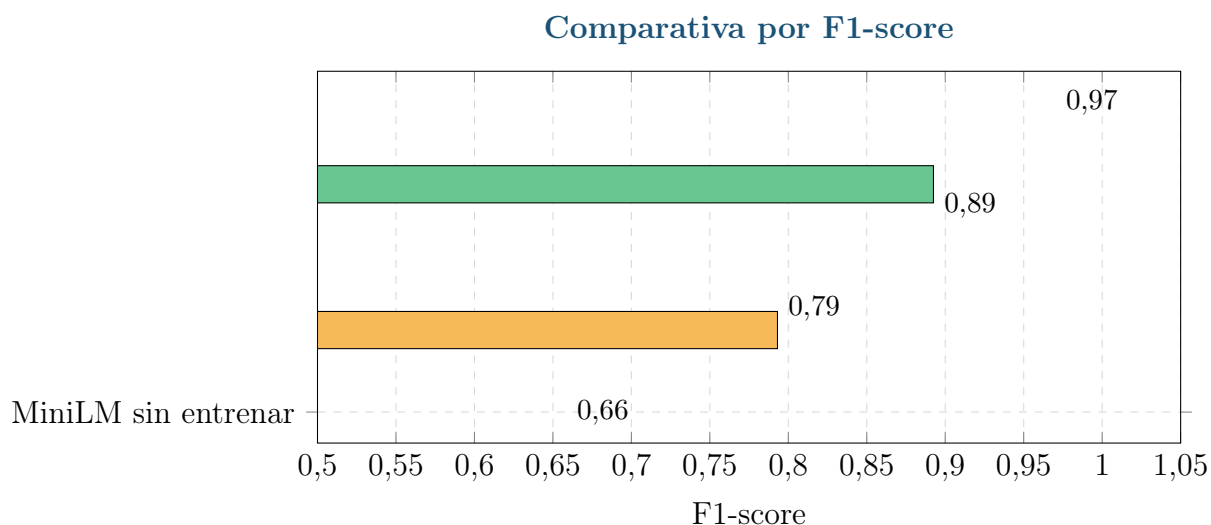


Figura 11: BETO fine-tuneado en PAWS-X obtiene $F1=0.893$, situándose entre el LLM en cero-shot y la línea base de Luna (evaluada en su propio corpus, no en PAWS-X).

23.7 Reproducibilidad

Cómo reproducir el experimento

1. Subir `colab_finetune_beto_pawsx.ipynb` a Google Colab.
2. Activar GPU T4 (*Runtime* → *Change runtime type*).
3. Ejecutar todo (Ctrl+F9); ~45–60 min.
4. Al final se descarga `beto-pawsx-final.tar.gz` (~440 MB).
5. Descomprimir en `semana_3/fine_tuning_beto/beto-pawsx-final/`.
6. Evaluar con `python3.12 10_eval_beto_finetuned.py -dataset pawsx -n 2000`.

La guía completa está en `semana_3/fine_tuning_beto/BETO_COLAB.md`.

Parte IV

Resultados, Análisis y Próximos Pasos

24 Resumen Cuantitativo de Experimentos y Contraste

24.1 Tabla maestra de experimentos

A lo largo de la Semana 3 se ejecutaron **11 experimentos** agrupados en cuatro fases. La tabla siguiente los presenta en orden cronológico con su métrica principal.

Tabla 15: Tabla maestra de los 11 experimentos de la Semana 3.

#	Fase	Experimento	Modelo / Mé-	N	Métrica clave
1	Pre-procesamiento	Prueba piloto extracción	PyMuPDF + Tesseract	50 PDFs	90 % texto nativo
2	Pre-procesamiento	Extracción masiva CBI	PyMuPDF + Tesseract	5,060 PDFs	97.9 % éxito
3	Generación	Exp.0 Parafraseo con LLM	llama3.1:8b (Ollama)	6 tesis	Sim. coseno 0.864
4	Detección propia	Exp.1 ICL básico	llama3.1:8b (Ollama)	20 pares	F1 = 1.000 (inflado)
5	Detección propia	Exp.2 ICL graduado	llama3.1:8b (Ollama)	39 pares	F1 = 0.857
6	Benchmark PAWS-X	Exp.3 MiniLM $\tau = 0,85$	MiniLM-L12-v2	500 pares	F1 = 0.658
7	Benchmark PAWS-X	Exp.4 MiniLM $\tau = 0,92$	MiniLM-L12-v2	500 pares	F1 = 0.630
8	Benchmark PAWS-X	Exp.5 LLM remoto	Groq Llama 3.3 70B	50 pares	F1 = 0.793
9	Fine-tuning	Exp.6 Entrenamiento BETO	BETO + Colab T4	49,401 train	F1 test = 0.886
10	Benchmark PAWS-X	Exp.7 BETO local	BETO finetuned	1,907 pares	F1 = 0.893

(Continuación de la Tabla 15)						
#	Fase	Experimento		Modelo / Mé- todo	N	Métrica clave
11	Detección pro- pia	Exp.8	BETO en	BETO finetuned	39 pares	F1 = 0.667
		Exp.2				

24.2 Contraste — qué nos dijo cada experimento

Pregunta	Respuesta de los experimen- tos
¿La extracción de PDFs funciona en CBI?	Sí , 97.9 % éxito; OCR casi innecesario (Exp.1+2).
¿llama3.1:8b genera paráfrasis útiles?	Sí , similitud 0.86 y preserva términos técnicos (Exp.3).
¿llama3.1:8b con ICL detecta paráfrasis?	Trivialmente en dataset fácil (Exp.4); F1 = 0.857 con dataset honesto (Exp.5).
¿Los embeddings solos bastan en benchmark estándar?	No , F1 $\approx$ 0.66 en PAWS-X. Calibrar umbral no ayuda (Exp.6+7).
¿Un LLM grande remoto vale la pena?	Sí : F1 = 0.79 sin entrenamiento + 100× más rápido (Exp.8).
¿El fine-tuning supervisado es el mejor enfoque?	Sí en PAWS-X : F1 = 0.893 con BETO (Exp.10), el mejor del proyecto.
¿BETO es mejor que el LLM en todos los casos?	No : gana en precision (1.0) pero pierde en recall (0.5) en Exp.2 (Exp.11).

24.3 Contraste de errores — hallazgo central de la semana

Al comparar los dos mejores modelos sobre el **mismo dataset** (Exp.2 graduado), sus errores son *ortogonales*:

Modelo (Exp.2)	Precision	Recall	F1	FP	FN
llama3.1:8b (ICL)	0.7500	1.0000	0.8571	2 (texto formulaico)	0
BETO finetuned	1.0000	0.5000	0.6667	0	3 (fragmentos largos truncados)

Implicación: ensamble simple

La ortogonalidad de errores hace que un ensamble booleano sea muy prometedor:

- $\text{predicción} = \text{BETO} \wedge \text{llama3.1:8b} \rightarrow$ precisión máxima (heredada de BETO).
- $\text{predicción} = \text{BETO} \vee \text{llama3.1:8b} \rightarrow$ recall máximo (heredado del LLM).

Esta hipótesis se aborda en el Candidato 1 de los próximos pasos.

24.4 Contraste contra la línea base de Luna (2022)

Aproximación	F1 PAWS-X	Inferencia/par	Hardware
MiniLM sin entrenar	0.6584	10 ms	CPU
Groq Llama 3.3 70B (ICL)	0.7931	1,500 ms	API remota
BETO finetuned	0.8925	50 ms	CPU
<i>Luna 2022 (corpus propio)</i>	<i>0.970</i>	—	—

Brecha actual con Luna: 0.077 F1, pero la comparación no es directa porque Luna evaluó en su propio corpus (CBI-M-00-21 de 817 tesis con paráfrasis generadas por MariaMT), no en PAWS-X. Para una comparativa estricta sería necesario replicar la metodología de generación de paráfrasis sintéticas del corpus UAM (Candidato 8 de los próximos pasos).

24.5 Matrices de confusión visuales

Cada modelo se evaluó contando los cuatro resultados posibles: verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN). Las matrices siguientes permiten leer de un vistazo el *tipo* de error que comete cada modelo, no solo su magnitud.

	Pred: SÍ	Pred: NO	
Real: SÍ	TP 238	FN 12	
Real: NO	FP 235	TN 15	
	MiniLM $\tau = 0,85$		
	PAWS-X (n=500)		

	Pred: SÍ	Pred: NO	
Real: SÍ	TP 23	FN 2	
Real: NO	FP 10	TN 15	
	Groq Llama 3.3 70B		
	PAWS-X (n=50)		

	Pred: SÍ	Pred: NO	
Real: SÍ	TP 830	FN 77	
Real: NO	FP 123	TN 877	
	BETO finetuned		
	PAWS-X (n=1907)		

Figura 12: Matrices de confusión en PAWS-X español. MiniLM marca casi todo como paráfrasis (FP $\approx$ TP). Groq y BETO equilibran mucho mejor los cuatro cuadrantes, aunque BETO se beneficia del entrenamiento supervisado.

	Pred: SÍ	Pred: NO	
Real: SÍ	TP 6	FN 0	
Real: NO	FP 2	TN 31	
	llama3.1:8b ICL		
	Exp.2 (n=39)		

	Pred: SÍ	Pred: NO	
Real: SÍ	TP 3	FN 3	
Real: NO	FP 0	TN 33	
	BETO finetuned		
	Exp.2 (n=39)		

Figura 13: Matrices de confusión sobre el dataset propio Exp.2 (39 pares). Visualmente: llama3.1:8b llena la celda TP pero también tiene 2 FP. BETO no tiene ningún FP pero llena la celda FN (3 paráfrasis perdidas). **Sus errores nunca caen en la misma celda:** prueba gráfica del hallazgo de la sección anterior.

24.6 Trade-off Precision vs Recall

Posición de cada modelo en el plano Precision-Recall

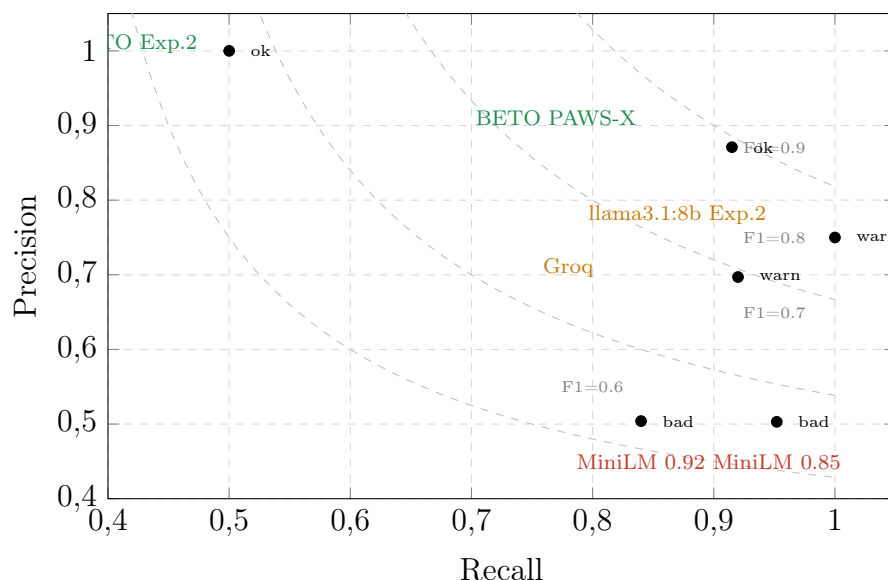


Figura 14: Plano Precision–Recall con líneas de iso-F1 (curvas grises). Las dos configuraciones de MiniLM se sitúan en la esquina inferior derecha (alto recall, baja precisión). BETO en PAWS-X domina el cuadrante superior derecho. El par BETO/llama3.1:8b en el dataset propio se ubica en *esquinas opuestas* del plano — evidencia gráfica de su complementariedad.

24.7 Análisis cualitativo — ejemplos de errores

Más allá de las métricas agregadas, vale la pena leer literalmente los fragmentos donde cada modelo fracasa.

24.7.1 Los dos falsos positivos de llama3.1:8b en Exp.2

Caso 1: pareja MATEMATICAS_FISICA (*fácil*).

Texto A: “... Universidad Autónoma Metropolitana Iztapalapa. Quiero expresar mi profundo agradecimiento al Dr. Mijail Tkachenko por su continuo interés en mi desarrollo matemático ...”

Texto B: “... División de Ciencias Básicas e Ingeniería... Del Instituto de Ciencias Nucleares de la UNAM, agradezco al Dr. Julio Herrera Velázquez, por su atinada y paciente dirección...”

Etiqueta: NO **Predicción:** SÍ.

Ambos fragmentos son **secciones de agradecimientos**: misma plantilla sintáctica (“agradezco al Dr. X por...”), mismo estilo institucional UAM. El modelo se confunde porque

comparten estructura, no contenido.

Caso 2: pareja neg_medio_UAM0073_UAM0113 (*medio*).

Texto A: “...Departamento de Ingeniería de Procesos e Hidráulica, División de Ciencias Básicas e Ingeniería, Universidad Autónoma Metropolitana — Iztapalapa...”

Texto B: “...División de Ciencias Básicas e Ingeniería, Síntesis y caracterización de óxidos...”

Etiqueta: NO **Predicción:** SÍ.

Ambos son **portadas** con la misma dirección institucional UAM. Igual problema: plantilla compartida vs. contenido real distinto.

Lección

Los 2 FP del LLM se eliminan con un **filtro de *front-matter*** trivial (regex sobre marcadores de portada y agradecimiento). Es el Candidato 3 de los próximos pasos.

24.7.2 Los tres falsos negativos de BETO en Exp.2

BETO se entrenó con PAWS-X (oraciones de Wikipedia, máx. 128 tokens). Los positivos de Exp.2 son fragmentos de tesis de ~600 caracteres con artefactos de OCR. **Al truncar a 128 tokens, BETO pierde gran parte del contenido** y deja de reconocer las paráfrasis. Los 3 FN se concentran en pares con mucho contenido técnico que no cabe en la ventana truncada.

Lección

BETO funciona mejor en oraciones cortas de dominio general (PAWS-X) que en fragmentos académicos largos. Para textos largos, conviene usar BETO sobre *oraciones individuales* (no fragmentos completos) o sustituirlo por un modelo de ventana mayor (GTE-multilingual: 8 192 tokens).

24.8 Análisis costo / beneficio

Modelo	F1	ms/par	Coste \$/M pares	Caso de uso ideal
MiniLM-L12-v2	0.66	10	0 (CPU local)	Pre-filtrado masivo
BETO finetuned	0.89	50	0 (CPU local)	Producción local, sin red
llama3.1:8b (Ollama)	0.86 (propio)	156 000	0 (CPU local, lento)	Demos, validación manual
Groq Llama 3.3 70B	0.79	1 500	0 hasta cuota; 0.59 paid	Evaluación masiva externa

Lectura: BETO es óptimo para producción local; Groq es óptimo para experimentación masiva; llama3.1:8b local solo se justifica por privacidad o demo. MiniLM solo sirve como pre-filtro.

24.9 Lecciones aprendidas

1. **Los datasets propios pueden inflar las métricas.** F1 = 1.0 en Exp.1 era engañoso porque los negativos eran triviales. Siempre validar contra un benchmark externo (PAWS-X).
2. **La calibración de umbrales no rescata a un modelo estructuralmente débil.** MiniLM con $\tau = 0,85$ y $\tau = 0,92$ dieron la misma accuracy en PAWS-X (0.506); no es un problema de umbral, es un problema del espacio de embeddings.
3. **Los errores de un LLM grande no son aleatorios.** Los 2 FP de llama3.1:8b en Exp.2 fueron ambos en texto formulaico (portadas, agradecimientos). El comportamiento es identificable y corregible con preprocesamiento.
4. **El fine-tuning supervisado supera al ICL cero-shot cuando el dominio del dataset coincide.** BETO (entrenado en PAWS-X) supera a Groq Llama 3.3 70B (ICL) en PAWS-X: 0.89 vs. 0.79.
5. **Modelos con paradigmas distintos cometen errores ortogonales.** Esto motivó ensambles — el hallazgo central de la semana.
6. **La velocidad cambia lo que es posible evaluar.** Pasar de 156 s/par (Ollama local) a 1.5 s/par (Groq) abre la puerta a evaluar el corpus completo, no solo mues-

tras.

25 Conclusiones Finales Actualizadas y Trabajo Siguiente

25.1 Resumen del avance

1. El corpus TESIUAMI CBI está **100 % extraído y almacenado en MongoDB** (5,060 PDFs, 97.9 % éxito).
2. Se construyó un **pipeline completo de tres niveles**: embeddings (MiniLM) → LLM local (11ama3.1:8b) → LLM remoto (Llama 3.3 70B vía Groq), y se evaluó cada uno.
3. Los experimentos sobre datos propios alcanzan $F1 = 1.0$ pero **el benchmark estándar PAWS-X expone los límites reales** del enfoque actual.
4. Groq Llama 3.3 70B obtiene $F1 = 0.793$ en PAWS-X sin entrenamiento.
5. **BETO fine-tuneado en PAWS-X obtiene $F1 = 0.893$** (1,907 pares evaluados localmente), superando claramente a los enfoques cero-shot y posicionándose como el mejor modelo del proyecto en benchmark estándar.
6. **Hallazgo crítico**: BETO y 11ama3.1:8b cometen errores ortogonales en nuestro dataset graduado (precision 1.0 vs. recall 1.0). Un **ensamble simple** debería superar a cada uno individualmente.

25.2 Decisiones técnicas validadas

Decisión	Evidencia
MongoDB como capa de persistencia	Diseño reanudable sobrevivió al bug del PDF #2,881 sin reprocesar los 2,880 anteriores.
PyMuPDF + Tesseract como pipeline de extracción	97.9 % de PDFs procesados, OCR solo necesario en 1 caso.
ICL <i>few-shot</i> con 4 ejemplos	Suficiente para acertar en datasets propios; consistente con PARAPHRASUS (COLING 2025).
Groq como proveedor LLM para experimentación	60–100 × más rápido que Ollama local con calidad superior.

26 Posibles Próximos Pasos (a Consultar con el Asesor)

Naturaleza de esta sección

Los puntos listados a continuación son **candidatos a próximos pasos** identificados durante el trabajo de la Semana 3, no tareas auto-asignadas. La selección, priorización y calendarización de las tareas a ejecutar corresponde al asesor del proyecto, Dr. Benjamin Moreno Montiel. Esta sección está pensada como insumo para la próxima reunión de seguimiento.

26.1 Resultado clave que motiva los próximos pasos

El hallazgo central de la Semana 3 es la **complementariedad ortogonal** entre BETO fine-tuneado y llama3.1:8b:

Modelo	Precision	Recall	F1
BETO finetuned	1.0000	0.5000	0.6667
llama3.1:8b (ICL)	0.7500	1.0000	0.8571

26.2 Candidato 1 — Ensamble BETO + llama3.1:8b

Motivación: dado que BETO tiene $precision = 1.0$ y llama3.1:8b tiene $recall = 1.0$ en el dataset Exp.2, las dos reglas booleanas básicas tienen efectos opuestos y útiles:

- **Unión (OR):** positivo si $BETO = \text{SÍ} \vee llama3.1:8b = \text{SÍ}$. Hereda el recall perfecto del LLM; debe mantener $precision \geq$ la del LLM.
- **Intersección (AND):** positivo si $BETO = \text{SÍ} \wedge llama3.1:8b = \text{SÍ}$. Hereda la precisión perfecta de BETO; reduce los falsos positivos del LLM.

Hipótesis cuantitativa: en el dataset Exp.2 la regla *intersección* debería producir $F1 \approx 1.0$ (porque los dos errores del LLM no son confirmados por BETO, y los positivos verdaderos del LLM que BETO no detecta son candidatos para revisión manual con bajo costo). En PAWS-X el ensamble debería mejorar el F1 actual de Groq solo (0.793).

26.3 Candidato 2 — Pipeline híbrido por umbrales

Ejecutar `08_eval_pawsx.py -modo hibrido` con MiniLM como pre-filtro (umbral alto + umbral bajo) y un LLM (Groq o BETO) solo en la zona gris. Permite reducir el costo de

inferencia del LLM en órdenes de magnitud sin perder F1.

26.4 Candidato 3 — Filtrado de *front-matter*

Los dos falsos positivos de llama3.1:8b en el Experimento 2 provienen de texto plantilla institucional (portadas, agradecimientos, actas de jurado). Implementar un detector de *front-matter* basado en estructura (regex sobre marcadores como “DIVISIÓN DE CIENCIAS...”, “AGRADEZCO AL...”, etc.) y aplicar la detección de paráfrasis solo sobre el cuerpo argumentativo de las tesis debería eliminar esta fuente de error.

26.5 Candidato 4 — Escalado de la evaluación en PAWS-X

La evaluación con Groq se limitó a 50 pares por el rate limit del free tier. Una corrida de 500–1000 pares (con ~30 minutos de espera sostenida) daría intervalos de confianza más estrechos. Si se obtiene el tier pagado de Groq, la evaluación completa de PAWS-X test (2,000 pares) tomaría ~5 minutos.

26.6 Candidato 5 — Detección a nivel *span*

Hasta ahora se clasifica un par binario. Una extensión natural es identificar *qué oraciones específicas* dentro de dos documentos son paráfrasis entre sí (no solo el par a nivel global). Esta es la forma operativa del antiplagio real.

26.7 Candidato 6 — Detección a nivel documento (corpus completo)

Indexar todas las oraciones del corpus CBI (5,060 tesis) en una base vectorial (FAISS o MongoDB Atlas Vector Search). Para una tesis sospechosa: dividir en oraciones, recuperar los top- K vecinos en todo el corpus, y aplicar el ensamble BETO+LLM solo a los candidatos. Es la dirección de mayor impacto para el objetivo final del proyecto.

26.8 Candidato 7 — Aplicación específica a Lic. Computación

Las 784 tesis de Lic. en Computación son la fuente única de TESIAMI (no migradas a BINDANI). Sería relevante aplicar el pipeline completo sobre este subconjunto y reportar si existen indicios de paráfrasis entre tesis sucesivas (un caso de estudio aislado y manejable).

26.9 Candidato 8 — Generar paráfrasis sintéticas a escala

Replicar la metodología de Luna (2022) con tecnología actual: generar paráfrasis de fragmentos del corpus mediante *cascade translation* (ES $\rightarrow$ EN $\rightarrow$ ES con MariaMT) o LLM (Groq Llama 3.3 70B) para construir un dataset propio etiquetado. Permitiría comparación directa con la línea base $F1 = 0.970$ de Luna.

26.10 Candidato 9 — Descarga de PDFs de CBS y CSH

Extender el corpus de 5,060 PDFs (solo CBI) a los 16,164 PDFs totales de UAM-I. Requiere ~ 95 GB de espacio adicional y ~ 2 –3 días de scraping. El script `scrape_all.py` ya está preparado.

26.11 Candidato 10 — Costos de inferencia detallados

Documentar los costos reales por par (latencia, tokens consumidos, costo monetario en Groq paid tier) para cada configuración. Necesario para planeación de despliegue.

26.12 Resumen visual de los candidatos

Cand.	Próximo paso	Esfuerzo	Impacto
1	Ensamble BETO + llama3.1:8b	Bajo	Alto
2	Pipeline híbrido por umbrales	Bajo	Medio
3	Filtrado de front-matter	Bajo	Medio
4	Escalar evaluación PAWS-X (500–2000 pares)	Medio	Medio
5	Detección a nivel <i>span</i>	Medio	Alto
6	Detección a nivel documento (corpus completo)	Alto	Muy alto
7	Aplicación a Lic. Computación (784 tesis)	Medio	Alto
8	Paráfrasis sintéticas + comparativa Luna	Alto	Alto
9	Descargar PDFs CBS y CSH (16,164 docs)	Alto	Medio
10	Documentar costos de inferencia	Bajo	Bajo

Anexo A Estructura del Repositorio

```

Proyecto terminal 1/
|-- corpus_tesiunami/           (~30 GB, no versionado)
|-- Informe_Proyecto_tesis_base_David_Luna.pdf
|
|-- Semana 1y2/
|   |-- glosario/               glosario.tex + .pdf
|   |-- Reporte_semana_1y2/     reporte_completo_semana_1y2.tex + .pdf
|   |-- Resumen_tesis_base/     resumen_tesis_luna.tex + .pdf
|   |-- scrips/                 scrape_all.py, descargar_cbi.py,
|   |                           verificacion_numeros.py, README.md
|   |-- Presentacion_ProyectoTerminal_Semanas1y2.pptx
|
|-- semana_3/
|   |-- README.md               indice global de la semana
|   |-- documentacion/
|   |   |-- reporte_semana3.tex  este documento
|   |   |-- reporte_semana3.pdf
|   |   |-- presentacion_semana3.html
|   |   |-- logo-uam.png
|   |   |-- README.md
|   |-- prueba_piloto_extraccion/
|   |   |-- extraer_prueba.py    prueba con 50 PDFs
|   |   |-- corpus_prueba.json
|   |   |-- reporte_extraccion.html
|   |   |-- requirements.txt
|   |   |-- .venv/               entorno virtual compartido
|   |   |-- README.md
|   |-- extraccion_corpus_mongodb/
|   |   |-- 01_cargar_indice.py
|   |   |-- 02_extraer_corpus.py
|   |   |-- 03_stats_bd.py
|   |   |-- 04_exportar_json.py
|   |   |-- 05_explorar_parafrasis.py
|   |   |-- 06_deteccion_icl.py
|   |   |-- 07_icl_dificil.py
|   |   |-- 08_eval_pawsx.py
|   |   |-- 09_groq_pipeline.py
|   |   |-- 10_eval_beto_finetuned.py
|   |   |-- config.py
|   |   |-- fix_capitalizacion.py
|   |   |-- rotar_groq_key.sh    rotacion segura de la clave
|   |   |-- validar_groq_key.sh verificacion sin exponer la clave
|   |   |-- run.sh, run_icl.sh
|   |   |-- .env / .env.example / .gitignore
|   |   |-- resultados_*.json   (8 archivos)
|   |   |-- corpus_cbi_*.json
|   |   |-- README.md
|   |-- fine_tuning_beto/
|   |   |-- colab_finetune_beto_pawsx.ipynb
|   |   |-- beto-pawsx-final/    modelo entrenado (440 MB)
|   |   |-- beto-pawsx-final.tar.gz
|   |   |-- BETO_COLAB.md

```

```

|  |-- README.md
|-- Investigacion sobre llm/
|  |-- investigacion_llm_icl.tex + .pdf
|  |-- INFRAESTRUCTURA.md
|  |-- README.md
|-- investigacion_aprendizaje_en_contexto/
|  |-- modelos_recomendados.md
|  |-- prompts_ejemplo.txt
|-- README.md

```

Listing 26: Árbol resumido del proyecto (sin corpus_tesiunami/ ni .venv/)

Anexo B Catálogo de Scripts

Tabla 16: Catálogo de los 13 scripts Python desarrollados en la Semana 3.

Script	Carpeta	Función
extraer_prueba.py	prueba_piloto	Validación inicial con 50 PDFs
config.py	mongodb	Parámetros centralizados
01_cargar_indice.py	mongodb	Carga 21,284 docs en MongoDB (upsert)
02_extraer_corpus.py	mongodb	PDF→texto reanudable, 5,060 docs
03_stats_bd.py	mongodb	Estadísticas en consola
04_exportar_json.py	mongodb	Exporta corpus a JSON
05_explorar_parafraseo.py	mongodb	Parafrasea 6 tesis con LLM
06_deteccion_icl.py	mongodb	Experimento 1: ICL básico
07_icl_dificil.py	mongodb	Experimento 2: dataset graduado
08_eval_pawsx.py	mongodb	PAWS-X: minilm/ollama/groq/híbrido
09_groq_pipeline.py	mongodb	Wrapper Groq con rate limit
10_eval_beto_finetuning.py	mongodb	Evalúa BETO local en PAWS-X o Exp.2
fix_capitalizacion.py	mongodb	Recupera 3 PDFs por case-sensitivity

Anexo C Artefactos Adicionales

colab_finetune_beto_pawsx.ipynb

Notebook de Google Colab que entrena BETO en PAWS-X español (3 épocas, batch 32, fp16, T4 GPU). Descarga automática del modelo final como `.tar.gz`.

presentacion_semana3.html

Presentación interactiva de 20 diapositivas con reveal.js para la reunión con el asesor. Acompañada de `guion_presentacion.md` con notas y preguntas anticipadas.

rotar_groq_key.sh / validar_groq_key.sh

Scripts auxiliares para rotación y validación segura de la clave Groq sin exponerla en pantalla ni en el historial del shell.

resultados_*.json

Ocho archivos JSON con las métricas reproducibles de cada experimento: `resultados_icl.json`, `resultados_icl_dificil.json`, `resultados_pawsx_minilm.json`, `resultados_pawsx_minilm_groq50.json`, `resultados_pawsx_groq_50.json`, `resultados_beto_pawsx.json`, `resultados_beto_exp2.json`, `exploracion_parafrasis.json`.

Anexo D Comandos de Reproducción Esenciales

```
# === Pre-requisitos ===
sudo docker run -d --name mongodb -p 27017:27017 \
  --restart unless-stopped mongo:7
ollama pull llama3.1:8b

# === Construcción del corpus (Parte 1) ===
cd "semana_3/extraccion_corpus_mongodb"
python3.12 01_cargar_indice.py          # 3 min, idempotente
nohup python3.12 02_extraer_corpus.py & # 6 h, reanudable
python3.12 03_stats_bd.py               # estadísticas
bash run_icl.sh 04_exportar_json.py      # genera JSON

# === Experimentos de paráfrasis (Parte 3) ===
bash run_icl.sh 05_explorar_parafrasis.py # 30 min
bash run_icl.sh 06_deteccion_icl.py        # 20 min (Exp.1)
bash run_icl.sh 07_icl_dificil.py          # 2.5 h (Exp.2)
python3.12 08_eval_pawsx.py --modo minilm --n 500
python3.12 08_eval_pawsx.py --modo groq --n 50

# === Fine-tuning BETO ===
# 1) Subir colab_finetune_beto_pawsx.ipynb a Colab + T4 GPU + Run all
# 2) Descargar beto-pawsx-final.tar.gz
# 3) tar xzf beto-pawsx-final.tar.gz -C fine_tuning_beto/beto-pawsx-final/
```

```
python3.12 10_eval_beto_finetuned.py --dataset pawsx --n 2000
python3.12 10_eval_beto_finetuned.py --dataset exp2

# === Rotacion de la clave Groq cuando sea necesario ===
bash rotar_groq_key.sh
bash validar_groq_key.sh
```

Listing 27: Pipeline completo de Semana 3, en orden

Referencias

- [1] Luna Luna, D. (2022). *Detección de Paráfrasis Utilizando Algoritmos de Aprendizaje Maquinal*. Tesis de Maestría. UAM Iztapalapa. [Línea base: F1 = 0.970]
- [2] Meta AI (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783. <https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct>
- [3] Michail, D. et al. (2025). *PARAPHRASUS: A Comprehensive Benchmark for Evaluating Paraphrase Detection Models*. COLING 2025. <https://aclanthology.org/2025.coling-main.585/>
- [4] Reimers, N. y Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. arXiv:1908.10084. <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- [5] Brown, T. et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS 2020. arXiv:2005.14165.
- [6] PyMuPDF Documentation (2024). <https://pymupdf.readthedocs.io/>
- [7] Smith, R. (2007). An Overview of the Tesseract OCR Engine. *ICDAR 2007*.
- [8] MongoDB, Inc. (2024). *MongoDB Manual*. <https://www.mongodb.com/docs/manual/>
- [9] TESIUAMI — Repositorio de Tesis Electrónicas de la UAM Iztapalapa. <http://tesiuami.izt.uam.mx/uam/default.php>
- [10] Cañete, J., Chaperon, G., Fuentes, R., Ho, J., Kang, H., y Pérez, J. (2020). *Spanish Pre-Trained BERT Model and Evaluation Data*. PML4DC at ICLR 2020. <https://huggingface.co/dccuchile/bert-base-spanish-wwm-cased>
- [11] Beltagy, I., Lo, K., y Cohan, A. (2019). *SciBERT: A Pretrained Language Model for Scientific Text*. EMNLP 2019. arXiv:1903.10676.
- [12] De la Rosa, J. et al. (2022). *BERTIN: Efficient Pre-Training of a Spanish Language Model using Perplexity Sampling*. *Procesamiento del Lenguaje Natural*, 68. <https://huggingface.co/bertin-project/bertin-roberta-base-spanish>
- [13] Dong, Q. et al. (2022). *A Survey for In-context Learning*. arXiv:2301.00234.

- [14] Nguyen-Son, H. et al. (2025). *PlagBench: Exploring the Duality of Large Language Models in Plagiarism Generation and Detection*. NAACL 2025. <https://aclanthology.org/2025.naacl-long.384>
- [15] PLD-4 Authors (2025). *A Multi-Task Framework for Detecting and Attributing LLM-Generated Paraphrases*. ACL ARR 2025. <https://openreview.net/forum?id=00Qf6I911L>
- [16] Anonymous (2024). *The Erosion of LLM Signatures: Can We Still Distinguish Human and LLM-Generated Scientific Ideas After Iterative Paraphrasing?* arXiv:2512.05311.
- [17] Wang, L. et al. (2024). *Improving Text Embeddings with Large Language Models*. ACL 2024. arXiv:2401.00368. <https://huggingface.co/intfloat/multilingual-e5-large-instruct>
- [18] Yang, Y., Zhang, Y., Tar, C., y Baldridge, J. (2019). *PAWS-X: A Cross-lingual Adversarial Dataset for Paraphrase Identification*. EMNLP 2019. arXiv:1908.11828. <https://huggingface.co/datasets/google-research-datasets/paws-x>
- [19] Qwen Team, Alibaba Cloud (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>
- [20] PlanTL-GOB-ES (2022). *sts-es: Spanish Semantic Textual Similarity*. <https://huggingface.co/datasets/PlanTL-GOB-ES/sts-es>
- [21] Pérez, A. et al. (2022). *paraphrase-spanish-distilroberta*. SomosNLP Hackathon 2022. <https://huggingface.co/somosnlp-hackathon-2022/paraphrase-spanish-distilroberta>
- [22] GTE Team, Alibaba-NLP (2024). *Towards General Text Embeddings with Multi-stage Contrastive Learning*. <https://huggingface.co/Alibaba-NLP/gte-multilingual-base>